



Professor André Duarte Bueno

<https://sites.google.com/view/professorandreduartebruno/>

email: bueno@lenep.uenf.br

UENF-CCT-LENEP-LDSC

26 de outubro de 2023

Copyright(C) André Duarte Bueno.

Todos os direitos reservados e protegidos pela Lei 5.988 de 14/12/1973. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor.

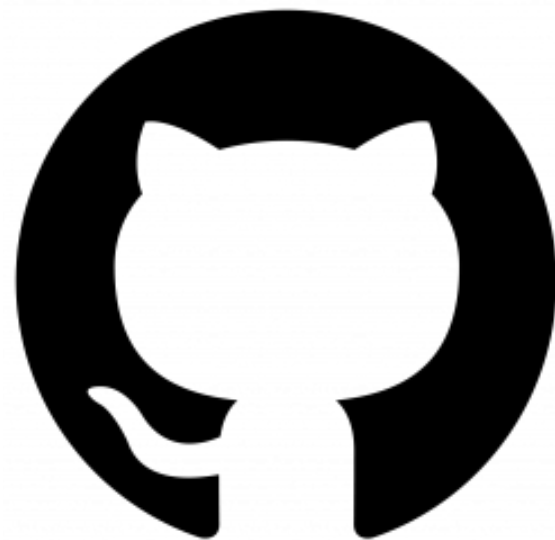
- *Editor:*

- *ANDRÉ DUARTE BUENO [Dr, Professor UENF/CCT/LENEP]*

- *Revisão técnica:*

- .

- Esta apostila foi desenvolvida na UENF/CCT/LENEP/LDSC
- Laboratório de Desenvolvimento de Software Científico - LDSC
 - <http://www.lenep.uenf.br/~ldsc>
- do Laboratório de Engenharia e Exploração de Petróleo - LENEP
 - <http://www.lenep.uenf.br>
- do Centro de Ciências e Tecnologia - CCT
 - <http://www.cct.uenf.br>
- da Universidade Estadual do Norte Fluminense - Darcy Ribeiro - UENF
 - <http://www.uenf.br>
- Site do Professor André Duarte Bueno
 - <https://sites.google.com/view/professorandreduartebruno/>



Sumário

1	Controle de Versões - O Programa <i>git</i>	12
1.1	Introdução ao <i>git</i>	14
1.1.1	Resumo histórico	14
1.1.2	O que é o <i>git</i> ?	15
1.1.3	O que o <i>git</i> não é?	17
1.1.4	Características do <i>git</i>	18
1.1.5	Versões do <i>git</i>	20
1.1.6	Como obter o <i>git</i> ?	21
1.1.7	Como obter informações/manuais do <i>git</i> ?	22
1.1.8	Como obter interfaces gráficas para o <i>git</i> ?	23

1.1.9	Sites com servidores públicos/privados do <i>git</i>	25
1.2	Como instalar e configurar o <i>git</i>	26
1.3	Principais comandos do <i>git</i>	31
1.3.1	Informações retornadas pelos comandos do <i>git</i>	33
1.4	Seqüência de trabalho <i>git</i> (um usuário)	35
1.4.1	Exemplo de uso do <i>git</i> com um usuário (linha de comando)	38
1.4.2	Exemplo de uso do <i>git</i> com um usuário (interface gráfica)	50
1.5	Como criar um repositório	51
1.5.1	Criando repositório vazio	51
1.5.2	Criando repositório a partir de diretório existente	52
1.6	Como baixar/clonar um repositório existente	53
1.6.1	Criando repositório a partir de repositório existente	53
1.6.2	Criando repositório a partir de modelo disponibilizado no <i>github</i> (<i>fork</i>)	53
1.7	Como modificar o repositório	54
1.7.1	Como atualizar os arquivos no repositório (<i>commit</i>)	54
1.7.2	Como adicionar/remover arquivos e diretórios	58
1.7.3	Como verificar as mudanças feitas usando <i>log</i>	66
1.8	Entendendo as versões - <i>git</i>	66

1.8.1	Como atualizar os arquivos no diretório de trabalho (<i>update</i>)	68
1.8.2	Como verificar diferenças entre arquivos	71
1.9	Desenvolvimento colaborativo <i>git</i>	74
1.10	<i>Tag's</i> e <i>releases</i>	77
1.10.1	Como criar <i>tag's</i> - versões de marcação	77
1.10.2	Como criar <i>release's</i> (versões de distribuição)	80
1.10.3	Como recuperar módulos e arquivos	84
1.11	Ramos e misturas (<i>branching</i> and <i>merging</i>)	86
1.11.1	Como trabalhar com ramos	87
1.11.2	Como mesclar duas versões de um arquivo	90
1.11.3	Como mesclar o ramo de trabalho com o ramo principal	91
1.12	Como verificar o estado do repositório	96
1.12.1	Como verificar o histórico das alterações	96
1.12.2	Como verificar as mensagens de <i>log</i> (<i>loginfo</i>)	97
1.12.3	Como verificar as anotações	100
1.12.4	Como verificar o <i>status</i> dos arquivos	101
1.13	Alguns exemplos de uso com vários usuários - Desenvolvimento em grupo	104
1.14	Um diagrama com os principais comandos do <i>git</i>	117

1.15	Definindo uma estratégia para trabalho em grupo	119
1.15.1	Fluxo de trabalho centralizado	120
1.15.2	Fluxo de trabalho baseado em funcionalidades	123
1.15.3	Todo - Doing - Done - Review - Aproved	125
1.16	Diferenças entre <i>cvs</i> , <i>svn</i> e <i>git</i>	126
1.17	Sentenças para o <i>git</i>	128
1.18	Resumo do capítulo	131
1.19	Exercícios	131

Referências Bibliográficas	133
----------------------------	-----

Lista de Figuras

1.1	Arquivo de configuração do <i>git</i>	30
1.2	Principais comandos do <i>git</i>	36
1.3	Principais comandos do <i>git</i> para atividade específica (ciclo para nova funcionalidade/característica).	37
1.4	Arquivo de configuração de repositório do <i>git</i>	49
1.5	<i>cvs</i> : Versões de um arquivo	67
1.6	<i>cvs</i> : Sequência de trabalho para desenvolvimento colaborativo	76
1.7	<i>cvs</i> : Como funciona um <i>tag</i>	78
1.8	<i>cvs</i> : Como funciona um <i>release</i>	83
1.9	<i>cvs</i> : Como ficam os ramos	86
1.10	<i>cvs</i> : Diagrama com os comandos do <i>git</i>	118

Lista de Tabelas

1.1	Saídas dos comandos do <i>cvs</i> e <i>flags</i> emitidos	34
1.2	Informações listadas pelo comando <i>cvs status</i>	102
1.3	Relação entre comandos do <i>cvs</i> do <i>svn</i> e do <i>git</i>	127

Listings

1.1	Saída do comando: <i>cv</i> s -H commit	54
1.2	Saída do comando: <i>cv</i> s commit após adição de um módulo	56
1.3	Saída do comando: <i>cv</i> s -H update	69
1.4	Saída do comando: <i>cv</i> s-diff	72
1.5	Saída do comando: <i>cv</i> s tag Etapa-1	79
1.6	Saída do comando: <i>cv</i> s commit -r 2	83
1.7	Exemplo: Mesclando dois ramos	94
1.8	Saída do comando: <i>cv</i> s history	97
1.9	Saída do comando: <i>cv</i> s log leiname.txt	97
1.10	Saída do comando: <i>cv</i> s annotate leiname.txt	100
1.11	Saída do comando: <i>cv</i> s status -v leiname.txt	103

Lista de programas

Capítulo 1

Controle de Versões - O Programa *git*

Neste capítulo veremos rapidamente as características do programa *git*.

- Introdução ao *git* (seção [1.1](#))
 - O que é o *git*? (seção [1.1.2](#)), como obter o *git*? (seção [1.1.6](#)).
- Características do *git* (seção [1.1.4](#)).
- Versões do *git* (seção [1.1.5](#)).
- Principais comandos do *git* ([1.3](#)).

- Configurando o *git* (??).
- Criando repositório no *git* (??).
- Desenvolvimento em grupo (??).
- Diferenças entre *cvs*, *svn* e *git* (seção [1.16](#)).

1.1 Introdução ao *git*

Veremos nesta seção um conjunto de definições relativas ao programa *git*.

1.1.1 Resumo histórico

1.1.2 O que é o *git*?

O *git* é um SCV do tipo distribuído de alta performance e segurança, é o padrão de fato da indústria na atualidade.

Como vimos, o *svn* é uma evolução incremental do *cvs*, com ele diversas melhorias foram adicionadas, mas o *svn* não é uma mudança de paradigma, ele funciona basicamente da mesma maneira que o *cvs*; Tanto o *cvs* quanto o *svn* usam o modelo cliente-servidor, neste modelo o usuário só tem a cópia de trabalho, não tendo acesso as informações se ocorrerem problemas com a rede internet. O *git*, desenvolvido pelo mesmo autor do GNU/Linux, Linus Torvards, usa uma abordagem diferente, cada cópia de trabalho é um repositório completo, com todas as informações do sistema, ou seja, é um sistema distribuído. Pode-se afirmar que o *git* é mais que uma evolução incremental. Para criar o *git*, Linus tomou como base quatro critérios:

- Tomar o *cvs* como um exemplo do que não fazer.
- Suportar um fluxo distribuído.
- Várias firmes proteções contra corrompimento de arquivos, seja por acidente ou origem maldosa.
- Alta performance.

Como curiosidade [wikipedia]:

“O desenvolvimento do Git começou em 3 de Abril de 2005.[9] O projeto foi anunciado em 6 de Abril,[10] e tornou-se auto-hospedeiro no dia 7 de Abril.[9] A primeira mescla de arquivos (merge) em múltiplos ramos (branches) foi realizado em 18 de Abril.[11] Torvalds alcançou seus objetivos de performance; em 29 de Abril, o recém-nascido Git se tornou referência ao registrar patches para a árvore de desenvolvimento do kernel do Linux na taxa de 6,7 por segundo.[12] No dia 16 de Junho, a entrega do kernel 2.6.12 foi gerenciada pelo Git.[13]”.

1.1.3 O que o *git* não é?

- O *git* não deve ser confundido como sendo um *software* para desenvolvimento (como uma IDE - interface de desenvolvimento).
- O *git* não substitui o gerenciamento do desenvolvimento do *software*, nem a necessidade de comunicação entre o grupo de desenvolvimento.
- O *git* não é uma ferramenta para testar o *software*.
- O *git* não tem um sistema para rastreamento de *bug*.

1.1.4 Características do *git*

Entre suas principais características podemos citar:

- Usa modelo distribuído, ponto a ponto.
- Cada instância de trabalho é uma cópia completa do repositório, permitindo trabalhar sem acesso a internet.
- Muito mais rápido que o *cvs* e *svn*.
- Comandos semelhantes aos do *cvs*.
- Criação de ramos com baixo custo.
- Multiplataforma com várias interfaces gráficas.
- Pode ou não ter um servidor centralizador.
- É mais difícil de usar, principalmente para novatos.

“Comparado com outros sistemas de controlo de versões tradicionais, o Git diferencia-se por ser extremamente rápido, por simplificar o desenvolvimento de software de forma não-linear, onde podemos trabalhar em

paralelo em diferentes funcionalidades das aplicações que desenvolvemos e então escolher quais funcionalidades devem fazer parte de cada versão da aplicação conforme o nosso fluxo de trabalho, e principalmente por ser um sistema distribuído bastante versátil e adequado para projectos de qualquer dimensão. ”[Proiete, 2011].

1.1.5 Versões do *git*

A primeira versão 1.0 do *git*, foi lançada em XX de XX de 20XX. Veremos a seguir as diferentes versões do *git* e suas características:

- Versão 1.0

- .

- Versão 1.1

- .

Veja no endereço [XXX](#), uma breve história sobre o *git*.

1.1.6 Como obter o *git*?

O programa *git* pode ser obtido nos endereços abaixo:

- <http://git-scm.com/>.

1.1.7 Como obter informações/manuais do *git*?

Uma documentação do *git* pode ser obtida num dos seguintes endereços:

- <http://git-scm.com/doc>.
- Git Wiki: https://git.wiki.kernel.org/index.php/Main_Page.
- Principais comandos: https://rogerdudler.github.io/git-guide/index.pt_BR.html.
- Guia prático: https://rogerdudler.github.io/git-guide/index.pt_BR.html

1.1.8 Como obter interfaces gráficas para o *git*?

- Interfaces gráficas:
 - <https://git-scm.com/downloads/guis>
 - https://git.wiki.kernel.org/index.php/Interfaces,_frontends,_and_tools#Yap.
- Git para Windows: <https://gitforwindows.org/>.
- Git para usuários do github: <https://desktop.github.com/>.
- Tortoise: <https://tortoisegit.org/download/>.
- qgit: <https://digilander.libero.it/mcostalba/#Download>, <https://sourceforge.net/projects/qgit/>.
- gitg: (<https://flathub.org/apps/details/org.gnome.gitg>).
- gitkraken: <https://www.gitkraken.com/>
- gitk:

Aplicativo educativo do uso do *git*:

- Site para teste/aprendizado dos comandos:
 - <https://git-school.github.io/visualizing-git/>.
 - <https://git-school.github.io/visualizing-git/#free>

1.1.9 Sites com servidores públicos/privados do *git*

Fonte: [Proiete, 2011].

Repositórios compartilhados pela rede.

- *GitHub*: <https://github.com/>
- *Gitorious*: <http://gitorious.org>
- *Unfuddle*: <http://unfuddle.com>
- *ProjectLocker*: <http://www.projectlocker.com>
- *RepositoryHosting*: <http://repositoryhosting.com>
- *Assembla*: <http://www.assembla.com>
- *Beanstalkapp*: <http://beanstalkapp.com/>
- *Sliksvn*: <http://www.sliksvn.com/>
- *Xp-dev*: <http://xp-dev.com>

1.2 Como instalar e configurar o *git*

1. Para instalar o *git*

(a) Se estiver usando uma distribuição GNU/Linux provavelmente já tem o *git* instalado. Use o comando

```
# git
```

para ver se já esta instalado.

Se ainda não estiver instalado use seu gerenciador de pacotes;

(a) Fedora:

```
# sudo dnf install git
```

(a) Ubuntu:

```
# sudo aptget install git
```

(a) Windows:

```
# ....
```

2. Configurações pessoais do *git* (são armazenadas no arquivo `~/.gitconfig`)

(a) Para definir o nome e e-mail do usuário:

```
# exemplo
# git config --global user.name "Seu Nome"
git config --global user.name "André Duarte Bueno"
# git config --global user.email "seu@email.google.com"
git config --global user.email "andreduartebruno@gmail.com"
```

(b) Para definir o nome do editor de texto:

```
git config --global core.editor "C:/Windows/notepad.exe"
```

ou no GNU/Linux

```
git config --global core.editor "emacs"
git config --global core.editor "vscode"
```

(c) Para ativar saída dos comandos com cores:

```
git config --global color.ui true
```

3. Da mesma forma que o *cvs*, o *git* permite que determinadas pastas e arquivos sejam ignorados. Isto pode ser útil por exemplo quando o compilador cria subdiretórios ocultos que você não quer adicionar ao repositório.

```
emacs .gitignore
# Ignorar os arquivos com as extensões
*.zip
*.tar.gz
*.obj
*.o
*~
# Ignorar os subdiretórios
obj/

git add .gitignore
git commit -m"Adicionado arquivo de controle das exclusões: .gitignore"

[bueno17@bueno-notebook workdir]$ git commit -m
"Adicionado arquivo de controle das exclusões: .gitignore"
# On branch master [main nas novas versões]
# Untracked files:
```

```
#   (use "git add <file>..." to include in what will be committed)
#
# .gitignore
nothing added to commit but untracked files present
(use "git add" to track)
```

Note que o *git* é um programa feito originalmente no GNU/Linux, logo, como a maioria dos programas GNU/Linux ele diferencia minúsculas de maiúsculas.

Dica: o site <https://github.com/github/gitignore> tem vários exemplos prontos de arquivos `.gitignore` para diferentes linguagens de programação.

4. Para obter uma ajuda dos comandos do *git* use:

```
git help nomeDoComando
```

5. Exemplos são encontrados em https://rogerdudler.github.io/git-guide/index.pt_BR.html, <https://git-scm.com/docs/git> e <https://docs.github.com/>.

A Figura 1.1 mostra um arquivo de configuração do *git*.


```
1 [user]
2     email = andreduartebruno@gmail.com
3     name = andreduartebruno
4 [credential "https://github.com"]
5     helper =
6     helper = !/usr/bin/gh auth git-credential
7 [credential "https://gist.github.com"]
8     helper =
9     helper = !/usr/bin/gh auth git-credential
10 [core]
11     editor = emacs
12 [color]
13     ui = true
-:--- .gitconfig All (1,0) (Conf[Unix] SP P
```

Figura 1.1: Arquivo de configuração do *git*

1.3 Principais comandos do *git*

Veja a seguir o protótipo do programa *git*.

Protótipo:

```
git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
  [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
  [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
  [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
  [--config-env=<name>=<envvar>] <command> [<args>]
```

Lista de opções do programa *git*

git

git - -help

Para informações sobre comando específico

git help <command>

Os principais comandos do *git* são o *git checkout*, que copia os arquivos do repositório para seu diretório de trabalho, o *git update*, que atualiza os arquivos do diretório de trabalho, e o *git commit*, que devolve ao repositório os arquivos que você modificou. Veja outros comandos do *git* na Tabela ??.

Comando		Descrição
<code>version</code>	X	Mostra versão do <i>git</i> .
<i>bisect</i>	N	<i>Find by binary search the change that introduced a bug</i>
<i>branch</i>	N	<i>List, create, or delete branches</i>
<i>clone</i>	N	<i>Clone a repository into a new directory</i>
<i>fetch</i>	N	<i>Download objects and refs from another repository</i>
<i>grep</i>	N	<i>Print lines matching a pattern</i>
<i>merge</i>	N	<i>Join two or more development histories together</i>
<i>mv</i>	N	<i>Move or rename a file, a directory, or a symlink</i>
<i>pull</i>	N	<i>Fetch from and merge with another repository or a local branch</i>
<i>push</i>	N	<i>Update remote refs along with associated objects</i>
<i>rebase</i>	N	<i>Forward-port local commits to the updated upstream head</i>
<i>reset</i>	N	<i>Reset current HEAD to the specified state</i>
<i>show</i>	N	<i>Show various types of objects</i>

1.3.1 Informações retornadas pelos comandos do *git*

Ao longo deste capítulo veremos exemplos de comandos do *git*, seus argumentos e a saída gerada.

Sempre que um comando do *git* é executado em um terminal, o *git* envia para a tela um conjunto de informações sobre as atividades realizadas. A Tabela 1.1 apresenta um conjunto de *flags* que costumam aparecer na tela e seu significado. Desta forma você vai poder analisar melhor a saída do *git*.

Tabela 1.1: Saídas dos comandos do *cvs* e *flags* emitidos

Flag	Descrição
N	Arquivo novo adicionado ao projeto.
P	O arquivo do diretório de trabalho esta atualizado.
U	O arquivo do diretório de trabalho esta atualizado.
C	O arquivo do repositório foi modificado e é necessário o comando <i>update</i> .
I	O arquivo foi ignorado pelo <i>cvs</i> .
L	O arquivo é um link simbólico.
A	O arquivo foi criado localmente, mas ainda não foi enviado para o repositório.
R	O arquivo foi localmente removido, mas a remoção ainda não foi notificada ao repositório.
M	O arquivo local foi modificado e é necessário o comando <i>commit</i> .
?	O arquivo existe somente no diretório de trabalho.

1.4 Seqüencia de trabalho *git* (um usuário)

A Figura 1.2 mostra um diagrama de sequência com os principais comandos do *git*.

A Figura 1.3 mostra um diagrama de sequência com os comandos do *git* usados para gerenciamento de uma atividade específica.

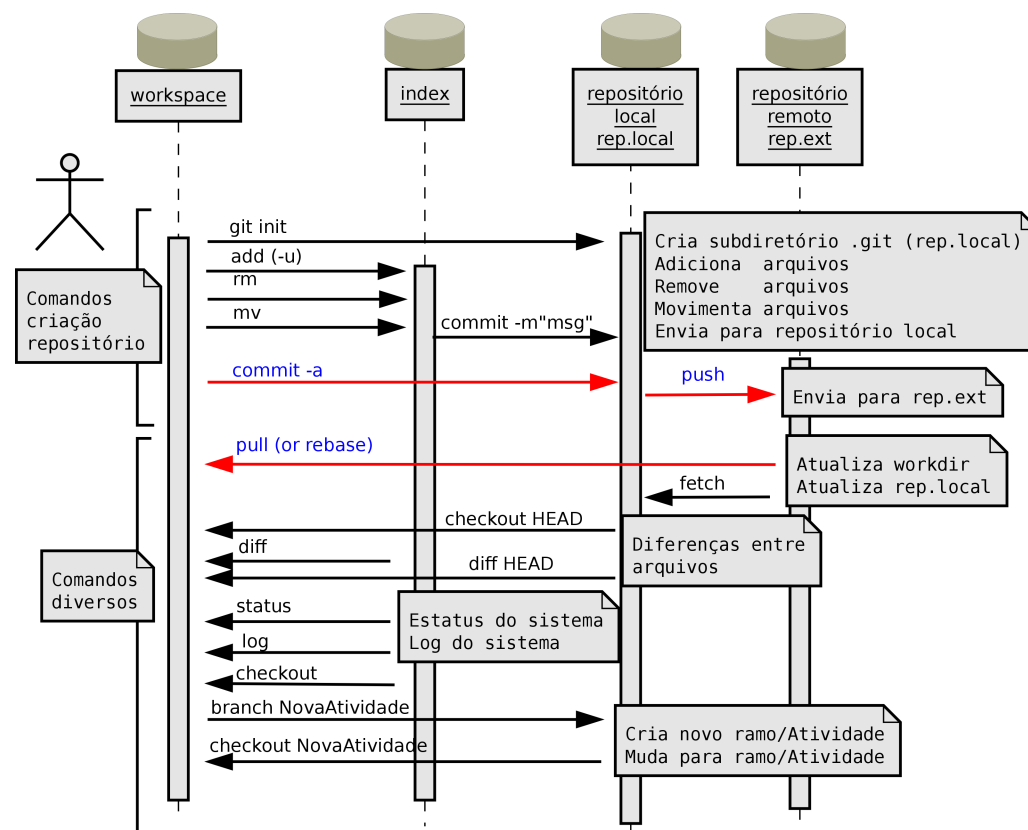


Figura 1.2: Principais comandos do *git*

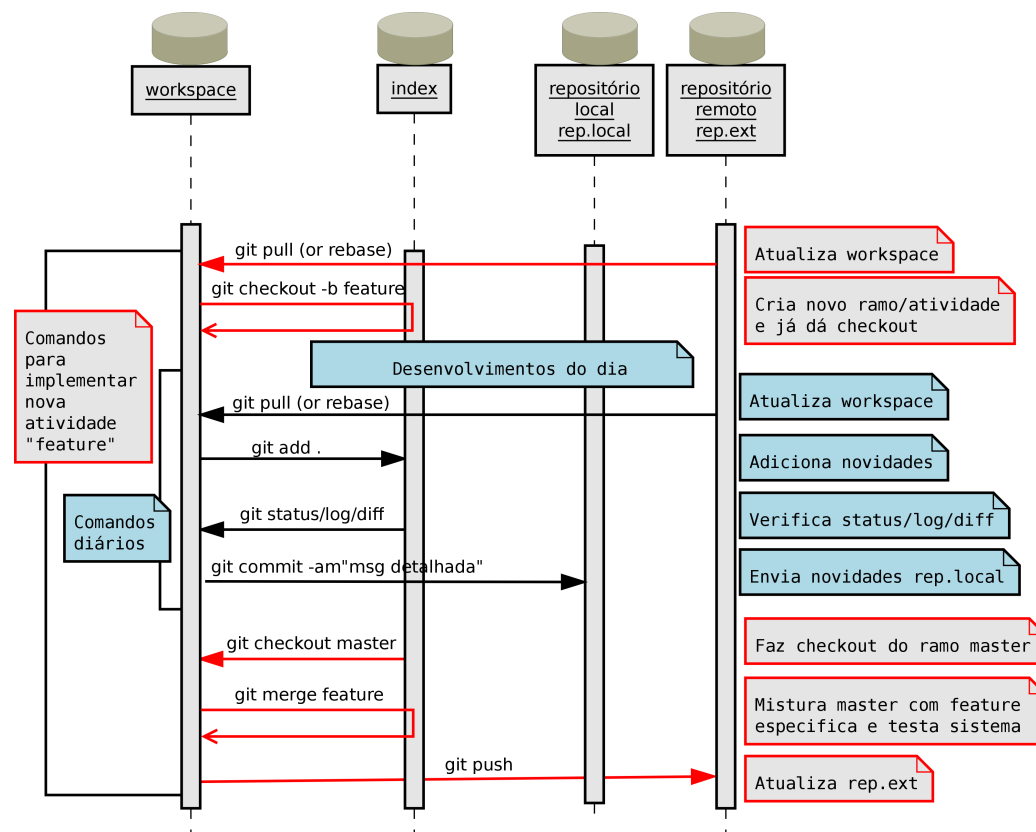


Figura 1.3: Principais comandos do *git* para atividade específica (ciclo para nova funcionalidade/característica).

1.4.1 Exemplo de uso do *git* com um usuário (linha de comando)

Apresenta-se aqui um exemplo de uso do *git* de forma bem simples e direta, a ideia é apresentar primeiro um exemplo prático e depois explicar os conceitos associados.

1. Para iniciar um repositório:

```
mkdir workdir
cd workdir
ls -lah workdir
git init

Initialized empty Git repository in /home/bueno17/Documentos/2-Ensino/
1-Apostilas-Livros-Pessoais/0-Livro-CPP/
LivroCPP-2nd-01-lyx-imagens-listagens
/listagens/Cap-42/git/workdir/.git/

ls -lah workdir

[bueno17@bueno-notebook workdir]$ ls -lah
total 12K
drwxrwxr-x. 3 bueno17 bueno17 4,0K Jun 12 19:30 .
```

```
drwxrwxr-x. 4 bueno17 bueno17 4,0K Jun 12 19:30 ..  
drwxrwxr-x. 7 bueno17 bueno17 4,0K Jun 12 19:30 .git
```

Observe que o *git* criou dentro do *workdir* um diretório oculto *.git* que é o repositório (local onde ficarão as diferentes versões).

2. Agora você pode criar arquivos.

```
cat > leiname.txt....ctrl+d  
cat > prog1.cpp....ctrl+d
```

3. Verifique o status de seu sistema.

```
git status  
[bueno17@bueno-notebook workdir]$ git status  
# On branch master  
#  
# Initial commit  
#  
# Untracked files:
```

```
# (use "git add <file>..." to include in what will be committed)
#
# leiname.txt
# prog1.cpp
# prog1.cpp~ nothing added to commit but untracked files present
# (use "git add" to track)
```

Observe que os arquivos ainda não fazem parte do repositório.

4. O *git* admite que é normal ter de fazer alterações por tarefas que envolvam um determinado grupo de arquivos. Assim, o usuário modifica o grupo de arquivos e dá um *git add* com estes arquivos, criando o que chamam de *"staging area"*. Um *"staging area"* é uma unidade lógica.

```
git add leiname.txt
git status

[bueno17@bueno-notebook workdir]$ git status
# On branch master
#
# Initial commit
```

```
#
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
# new file:   leiame.txt
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# prog1.cpp
# prog1.cpp~
git add prog1.cpp
git status

[bueno17@bueno-notebook workdir]$ git status
# On branch master
#
# Initial commit
#
```

```
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
# new file:   leiname.txt
# new file:   prog1.cpp
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# prog1.cpp~
```

5. Posteriormente as mudanças do workdir são enviadas para o repositório com um *commit*.

```
git commit -m"Mensagem associada ao commit;
adicionados arquivos leiname.txt e prog1.cpp"
[master (root-commit) bb048be] Mensagem associada ao commit;
adicionados arquivos leiname.txt e prog1.cpp
2 files changed, 9 insertions(+)
create mode 100644 leiname.txt
```

```
create mode 100644 prog1.cpp
git status
[bueno17@bueno-notebook workdir]$ git status
# On branch master
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# prog1.cpp~
nothing added to commit but untracked
files present (use "git add" to track)
```

"Idealmente cada *commit* representa uma unidade lógica, como por exemplo uma nova funcionalidade implementada numa aplicação, ou uma correcção de um *bug*, ou seja, um *commit* é um conjunto de alterações que foram feitas no repositório, e podem ser alterações em ficheiros existentes, criação de novos ficheiros, ou remoção de um ou mais ficheiros." [Proiete, 2011].

6. Você pode modificar o arquivo `leiametext`, criar um arquivo novo e a seguir verificar o estado do *workdir*.

```
echo "Conteúdo Diretório: " >> leiametext
```

```
ls >> leiame.txt
cat >> prog2.cpp....ctrl+d
git status

[bueno17@bueno-notebook workdir]$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..."
# to discard changes in working directory)
#
# modified:   leiame.txt
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# prog1.cpp~
# prog2.cpp
# prog2.cpp~
```

no changes added to commit (use "git add" and/or "git commit -a")

7. Pode adicionar vários arquivos usando:

```
git add .
```

Note que todos os arquivos são adicionados.

8. Adicione as mudanças no repositório:

```
git commit -m"Adicionado arquivo prog2.cpp"
[master bbefa47] Adicionado arquivo prog2.cpp
4 files changed, 29 insertions(+)
create mode 100644 prog1.cpp~
create mode 100644 prog2.cpp
create mode 100644 prog2.cpp~
[bueno17@bueno-notebook workdir]$ emacs .gitignore
Gtk-Message: Failed to load module "pk-gtk-module"
[bueno17@bueno-notebook workdir]$ git commit
-m"Adicionado arquivo de controle das exclusões: .gitignore"
```



```
# On branch master
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# .gitignore
nothing added to commit but untracked files present (use "git add" to track)
```

Note que se esquecer de passar a opção -m, o editor padrão é aberto; neste caso basta digitar a mensagem e salvar o arquivo.

9. Neste momento já fizemos alguns *commits* seria interessante verificar o log destes *commits*. Note que o comando *git log*, retorna o identificador do *commit*, o nome do autor, a data de modificação e a mensagem do *commit*.

```
git log
commit bbefa477a700ebc5420e14d8624c12ee7c9bc890
Author: André Duarte Bueno <andreduartebruno@gmail.com>
Date:   Tue Jun 12 19:40:01 2012 -0300
    Adicionado arquivo prog2.cpp
commit bb048be11ba8478be86e6525d23eb931dd626719
```

Author: André Duarte Bueno <andreduartebruno@gmail.com>

Date: Tue Jun 12 19:37:18 2012 -0300

Mensagem associada ao commit; adicionados arquivos leiam.txt e prog1.cpp

Nota: no caso específico do pacote *msysGit*, tem-se como alternativa o comando *gitk*, que gera uma saída gráfica.

commit bb048be11ba8478be86e6525d23eb931dd626719

Author: André Duarte Bueno <andreduartebruno@gmail.com>

Date: Tue Jun 12 19:37:18 2012 -0300

Mensagem associada ao commit; adicionados arquivos leiam.txt e prog1.cpp

Para maiores detalhes deste comando execute (o mesmo é válido para outros comandos):

`git log --help`

GIT-LOG(1)

Git Manual

GIT-LOG

NAME

git-log - Show commit logs

SYNOPSIS

```
git log [<options>] [<since>..

DESCRIPTION



Shows the commit logs.



The command takes options applicable to the



git rev-list command



to control what is



shown and how, and options applicable to the git diff-*



commands to control how the



changes each commit introduces are shown.

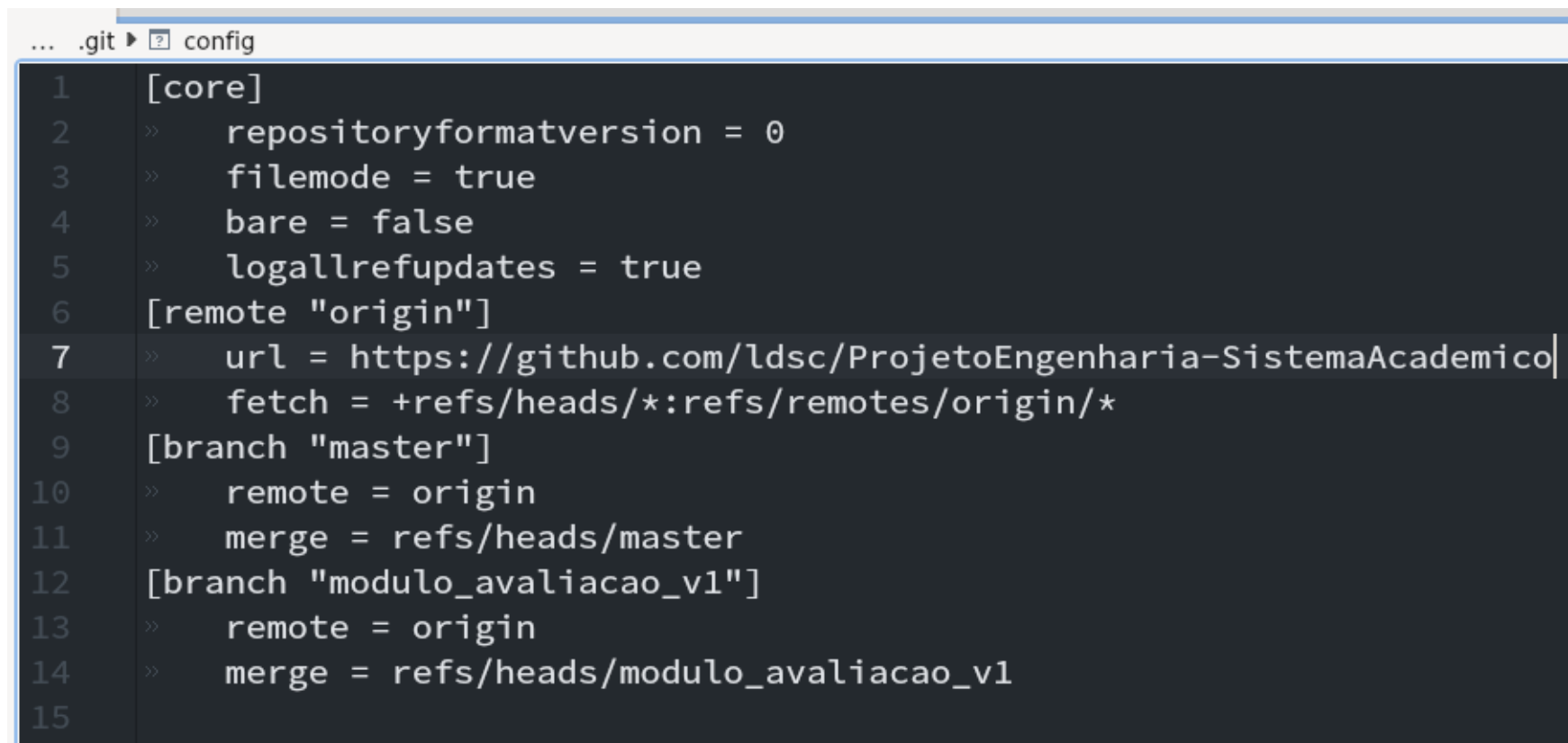


...continua...


```

Dica: Podemos usar o comando `git commit -a` para realizar um commit sem a necessidade de adicionar os arquivos modificados.

A Figura [1.4](#) mostra um arquivo de configuração de um repositório do `git`.

A screenshot of a code editor showing the contents of a .git/config file. The file is named 'config' and is located within a directory named '.git'. The code is written in a dark-themed editor with line numbers on the left. The configuration includes settings for the core, a remote named 'origin', and two branches: 'master' and 'modulo_avaliacao_v1'.

```
1 [core]
2 » repositoryformatversion = 0
3 » filemode = true
4 » bare = false
5 » logallrefupdates = true
6 [remote "origin"]
7 » url = https://github.com/ldsc/ProjetoEngenharia-SistemaAcademico|
8 » fetch = +refs/heads/*:refs/remotes/origin/*
9 [branch "master"]
10 » remote = origin
11 » merge = refs/heads/master
12 [branch "modulo_avaliacao_v1"]
13 » remote = origin
14 » merge = refs/heads/modulo_avaliacao_v1
15
```

Figura 1.4: Arquivo de configuração de repositório do *git*

1.4.2 Exemplo de uso do *git* com um usuário (interface gráfica)

Vamos repetir o mesmo procedimento visto acima, mas agora usando uma interface gráfica.

- *qgit*: <https://digilander.libero.it/mcostalba/#Download>, <https://sourceforge.net/projects/qgit/>.
- *gitg*: (<https://flathub.org/apps/details/org.gnome.gitg>).
- *gitkraken*: <https://www.gitkraken.com/>
- *gitk*:

Agora que já vimos um exemplo passo a passo bem direto, vamos apresentar cada comando com mais detalhes.

1.5 Como criar um repositório

1.5.1 Criando repositório vazio

A ideia aqui é criar um diretório novo e definir este diretório como sendo um repositório do *git*.

- Comandos

```
cd ~/workdir
mkdir nomeRepositorio
cd nomeRepositorio
git init
```

1.5.2 Criando repositório a partir de diretório existente

A ideia aqui é transformar um diretório existente num repositório do git.

- Comandos

```
cd ~/workdir/  
mv caminhoParaRepositorioExistente ~/workdir/  
cd ~/workdir/caminhoParaRepositorioExistente  
git init
```

1.6 Como baixar/clonar um repositório existente

1.6.1 Criando repositório a partir de repositório existente

A ideia aqui é fazer uma cópia de um repositório existente que já está na minha máquina, ou seja, clonar um repositório local.

- Comandos

```
cd ~/workdir/  
git clone caminhoParaORepositorioExistente NomeNovoRepositorio
```

1.6.2 Criando repositório a partir de modelo disponibilizado no *github* (*fork*)

- Comandos

```
cd ~/workdir  
git clone https://github.com/ldsc/ModeloDocumento-ProjetoEngenharia-ProgramacaoPratica
```


1.7 Como modificar o repositório

DAQUI PARA BAIXO PRECISA GERAR PARA GIT - aqui....

1.7.1 Como atualizar os arquivos no repositório (*commit*)

O comando *cvs commit* é utilizado para atualizar os arquivos no repositório, isto é, copiar para o repositório todos os arquivos novos ou modificados (inclusive os sub-diretórios).

Para obter uma listagem das opções do comando *commit*, abra um terminal e execute o comando ***cvs -H commit***. Veja saída na listagem 1.1.

Listing 1.1: Saída do comando: ***cvs -H commit***

```
1 [bueno@bueno cvs]$ cvs -H comit
2 Unknown command: 'comit'
3
4 CVS commands are:
5     add          Add a new file/directory to the repository
6     admin        Administration front end for rcs
7     annotate      Show last revision where each line was modified
8     checkout     Checkout sources for editing
9     commit       Check files into the repository
```

```

10      diff          Show differences between revisions
11      edit          Get ready to edit a watched file
12      editors       See who is editing a watched file
13      export        Export sources from CVS, similar to checkout
14      history       Show repository access history
15      import        Import sources into CVS, using vendor branches
16      init          Create a CVS repository if it doesn't exist
17      log           Print out history information for files
18      login         Prompt for password for authenticating server
19      logout        Removes entry in .cvspass for remote repository
20      pserver       Password server mode
21      rannotate     Show last revision where each line of module was modified
22      rdiff         Create 'patch' format diffs between releases
23      release       Indicate that a Module is no longer in use
24      remove        Remove an entry from the repository
25      rlog          Print out history information for a module
26      rtag          Add a symbolic tag to a module
27      server        Server mode
28      status        Display status information on checked out files
29      tag           Add a symbolic tag to checked out version of files
30      unedit        Undo an edit command
31      update        Bring work tree in sync with repository
32      version       Show current CVS version(s)
33      watch         Set watches

```

```

34 watchers See who is watching a file
35 (Specify the --help option for a list of other help options)

```

Veja na listagem 1.2 a saída do comando *cvs commit* executada no diretório de trabalho pelo usuário_1 após a modificação do arquivo CVSROOT/modules. Observe que o cvs mudou o número da versão do arquivo modules, que passou de 1.1 para 1.2. Após o *commit* o arquivo do repositório ficará igual ao arquivo do diretório de trabalho.

Listing 1.2: Saída do comando: ***cvs commit*** após adição de um módulo

```

1 [bueno@bueno exemplo-biblioteca-gnu]$ cvs commit -m "adicionado apelido lib-gnu"
2 cvs commit: Examining .
3 cvs commit: Examining CVSROOT
4 Checking in CVSROOT/modules;
5 /home/REPOSITORIO/CVSROOT/modules,v <-- modules
6 new revision: 1.2; previous revision: 1.1
7 done
8 cvs commit: Rebuilding administrative file database

```

Como criamos um apelido para o projeto (lib-gnu) podemos executar o comando *cvs checkout* de forma abreviada, utilizando o apelido criado (o módulo criado) e a forma abreviada de *checkout* (veja Tabela ??). A seguir um segundo usuário copia os arquivos do repositório para seu diretório de trabalho.

```

mkdir /tmp/usuario_2
cd /tmp/usuario_2

```

cvs co lib-gnu

cvs checkout: Updating lib-gnu

U lib-gnu/CPonto.h

U lib-gnu/CPonto.cpp

U lib-gnu/ProgCPonto.cpp

Observe que o nome do diretório de trabalho obtido pelo usuário_1 é exemplo-biblioteca-gnu e do usuário_2, lib-gnu. Isto é, se você utiliza *cvs checkout* nome_projeto, o *cvs* cria o diretório nome_projeto. Se você usa *cvs checkout nome_módulo*, o *cvs* cria o diretório nome_modulo.

1.7.2 Como adicionar/remover arquivos e diretórios

O comando *cvs add* agenda a adição de arquivos e diretórios que só serão copiados para o repositório com o comando *cvs commit*. Da mesma forma, o comando *cvs remove* agenda a remoção de arquivos e diretórios que só serão removidos do repositório com o comando *cvs commit*.

Veja a seguir o protótipo desses comandos. Observe que para os comandos funcionarem, você deve estar no diretório de trabalho.

Protótipo:

cvs add arquivos

cvs add diretórios

cvs remove arquivos

cvs remove diretórios

Como adicionar um arquivo

Vamos criar um arquivo *leiametext*, que contém alguma informação sobre o projeto. Vamos criá-lo com o editor *emacs* (use um editor que você conhece). Depois vamos agendar a adição do arquivo criado com o comando *cvs add*:

cd Usuário_1

emacs leiametext

...inclua informações sobre o projeto...

cvs add leiname.txt

cvs add: scheduling file 'leiname.txt' for addition

cvs add: use 'cvs commit' to add this file permanently

Observe a mensagem gerada pelo *cvs*, para adicionar o arquivo de forma permanente (isto é, no repositório) precisamos executar o comando *cvs commit*.

cvs commit -m “adicionado arquivo leiname.txt”

cvs commit: Examining .

cvs commit: Examining CVSROOT

RCS file:/home/REPOSITORIO/exemplo-biblioteca-gnu/leiname.txt,v done

Checking in leiname.txt;

/home/REPOSITORIO/exemplo-biblioteca-gnu/leiname.txt,v <--leiname.txt

initial revision: 1.1

done

Observe na saída acima que o *cvs* examina os sub-diretórios de forma recursiva, encontra o arquivo *leiname.txt* e o envia para o repositório definindo um novo número de versão (1.1).

Alguns comandos do programa *cvs* podem abrir um editor de texto para que você inclua alguma mensagem relativa à operação que foi realizada. No exemplo anterior, se tivéssemos digitado *cvs commit*, o *cvs* iria abrir o editor padrão do sistema¹. Após a abertura do editor padrão, digite uma mensagem informativa relativa as modificações que foram realizadas. Observe que com o uso de *cvs commit -m "mensagem"* evita-se a abertura do editor padrão, pois você já está passando a mensagem.

Como adicionar vários arquivos:

O procedimento é igual ao anterior mas em vez de passar o nome de um arquivo passamos um *wildcard* (ex: *, ?); primeiro, use o comando *cvs add* para agendar a adição dos arquivos, depois, adicione efetivamente os arquivos com o comando *cvs commit*. Observe que podemos informar o *cvs* que determinado arquivo é binário usando a opção *-kb* (arquivos adicionados como binários *-kb*, vão para o repositório, mas não são processados pelo *cvs*).

```
cvs add -m "adicionando arquivos de texto" *.txt
cvs add -kb -m "adicionando arquivos binários do word" *.doc
cvs commit
```

¹Na sua máquina provavelmente irá abrir o editor vi (ou notepad). No vi, digite esc:q, para sair, e esc:q!, para sair sem salvar alterações. Você pode alterar o editor a ser aberto pelo *cvs*, definindo no arquivo ~/.bash_profile a variável de ambiente CVSEDITOR (em minha máquina: export CVSEDITOR=emacs).

Como adicionar um diretório:

A seqüência envolve a criação do diretório, o agendamento da adição e a efetiva adição do diretório com *cvs commit*:

```
mkdir novoDiretório  
cvs add novoDiretório  
cvs commit -m "adicionado novo diretório"
```

Como adicionar toda uma estrutura de diretórios num projeto existente:

É o mesmo procedimento utilizado para importar todo um projeto. A única diferença é que o caminho para o projeto no repositório será relativo ao caminho de um projeto existente. Veja o protótipo:

```
cd sub_diretório_do_projeto  
cvs import -m "mensagem" nome_projeto/sub_diretório_do_projeto release tag
```

Como remover um arquivo:

Se você executar o comando *cvs remove arquivo*, sem que o arquivo tenha sido localmente removido, o *cvs* envia uma mensagem de erro que pede para você remover o arquivo antes de submeter a remoção ao repositório. Ou seja, você deve remover o arquivo do diretório de trabalho, deve agendar a remoção e, então, efetivar a remoção com *cvs commit*:

cvs remove leiame.txt

cvs remove: file 'leiame.txt' still in working directory

cvs remove: 1 file exists; remove it first

rm leiame.txt

cvs remove leiame.txt

cvs remove: scheduling 'leiame.txt' for removal

cvs remove: use 'cvs commit' to remove this file permanently

cvs commit -m "removido leiame.txt"

cvs commit: Examining .

cvs commit: Examining CVSROOT

Removing leiame.txt;

/home/REPOSITORIO/exemplo-biblioteca-gnu/leiame.txt,v <-- leiame.txt

new revision: delete; previous revision: 1.1

done

O comando a seguir remove o arquivo no diretório de trabalho e no repositório ao mesmo tempo:

cvs remove -f leiname.txt

Um arquivo que tenha sido removido (talvez acidentalmente) pode ser ressuscitado com um comando *cvs add*. Veja o exemplo:

cvs add leiname.txt

cvs add: Resurrecting file 'leiname.txt' from revision 1.1.

U leiname.txt

cvs add: Re-adding file 'leiname.txt' (in place of dead revision 1.2).

cvs add: use 'cvs commit' to add this file permanently

cvs ci -m "recuperei leiname.txt"

cvs commit: Examining .

cvs commit: Examining CVSROOT

Checking in leiname.txt;

/home/REPOSITORIO/exemplo-biblioteca-gnu/leiname.txt,v <-- leiname.txt

new revision: 1.3; previous revision: 1.2

done

Se você fizer alterações em um arquivo e depois removê-lo, não poderá recuperá-las. Para que possa recuperar as alterações, você deve criar uma versão do arquivo no repositório utilizando os comandos *cvs add* e *cvs commit*.

Dica: Os arquivos removidos são movidos para um sub-diretório do repositório chamado “*attic*” (sótão).

Como remover vários arquivos:

Você deve remover os arquivos, agendar a remoção e então remover do repositório com *cvs commit*.

```
rm -f *.txt  
cvs remove  
cvs commit -m "removidos diversos arquivos *.txt"
```

Como remover diretórios:

Vá para dentro do diretório que quer remover e apague todos os arquivos e o diretório utilizando:

```
cd nome_do_diretório  
cvs remove -f *  
cvs commit -m "removidos arquivos"
```

A seguir delete o diretório:

```
cd ..  
rmdir nome_do_diretório  
cvs remove nome_do_diretório/  
cvs commit -m "removidos diretórios"
```

Como renomear arquivos:

Vá para dentro do diretório onde está o arquivo a ser renomeado, renomeie o arquivo com *mv*, remova o arquivo antigo do repositório e adicione o novo, veja exemplo:

```
cd diretorio  
mv nome_antigo nome_novo  
cvs remove nome_antigo  
cvs add nome_novo  
cvs commit -m "Renomeado nome_antigo para nome_novo"
```

Nota: Observe que o arquivo é removido e um novo é adicionado. O programa *git*, apresentado no Capítulo ?? - Controle de Versões - O *github*, permite que os arquivos e diretórios sejam efetivamente renomeados.

1.7.3 Como verificar as mudanças feitas usando log

O comando `git log` permite ver o histórico das alterações nos arquivos.

```
$ git log
```

Para ver as diferenças de forma detalhadas use

```
$ git log -p
```

Também podemos obter um sumário das alterações

```
$ git log --stat --summary
```

1.8 Entendendo as versões - *git*

Todos os arquivos do projeto que foram importados ou adicionados no repositório têm um número de versão. A versão é definida automaticamente pelo programa *cvs* e se aplica aos arquivos individualmente, isto é, cada arquivo tem sua versão.

De uma maneira geral, a versão do arquivo é redefinida a cada alteração que foi comutada com o repositório (observe as saídas dos comandos *cvs commit*).

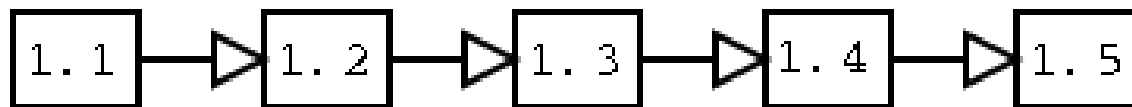


Figura 1.5: *cvs*: Versões de um arquivo

No exemplo a seguir, o usuário_2 vai para o diretório de trabalho e modifica o arquivo *leiametext*. Depois, um *cvs commit* é realizado. Observe que como o arquivo *leiametext* foi alterado, quando o mesmo for atualizado no repositório, o mesmo passará a ter a versão 1.4. Veja Figura 1.5.

```
cd /tmp/usuario_2/lib-gnu  
cvs update
```

```
cvs update: Updating . U leiametext
```

```
emacs leiametext
```

```
...alterações feitas pelo usuário 2...
```

```
cvs commit -m "atualização arquivo leiametext"
```

```
cvs commit: Examining.  
Checking in leiame.txt;  
/home/REPOSITORIO/exemplo-biblioteca-gnu/leiame.txt,v <-- leiame.txt  
new revision: 1.4; previous revision: 1.3  
done
```

1.8.1 Como atualizar os arquivos no diretório de trabalho (*update*)

Como o *cvs* permite o trabalho em grupo, um segundo usuário pode ter copiado e alterado os arquivos do projeto no repositório (veja Figura ??). Neste caso, alguns arquivos com os quais você está trabalhando podem estar desatualizados. Isto é, se um outro usuário modificou algum arquivo do repositório, você precisará atualizar os arquivos em seu diretório de trabalho.

Bastaria realizar um comando *cvs commit*, para enviar para o repositório os arquivos que você modificou, e um comando *cvs checkout*, para copiar os arquivos do repositório, atualizados, para seu diretório de trabalho. Mas este procedimento pode ser lento. Seria mais rápido se o programa *cvs* copiasse para o seu diretório de trabalho apenas os arquivos novos e modificados. É exatamente isto que o comando *cvs update* faz. Abra um terminal e execute o comando ***cvs -H update*** para obter uma lista com as opções do comando *update* (veja listagem [1.3](#)).

Protótipo:

```
# Para atualizar os arquivos do diretório de trabalho
```

```
cvs update  
# Para atualizar os arquivos a partir de uma data específica  
cvs update -D "21/5/2012"  
cvs update -D "21 Apr"  
cvs update -D "1 year ago"
```

Listing 1.3: Saída do comando: *cvs -H update*

```
1 [bueno@bueno cvs]$ cvs -H update  
2 Usage: cvs update [-APCdfIRp] [-k kopt] [-r rev] [-D date] [-j rev]  
3      [-I ign] [-W spec] [files...]  
4      -A      Reset any sticky tags/date/kopts.  
5      -P      Prune empty directories.  
6      -C      Overwrite locally modified files with clean repository copies.  
7      -d      Build directories, like checkout does.  
8      -f      Force a head revision match if tag/date not found.  
9      -l      Local directory only, no recursion.  
10     -R      Process directories recursively.  
11     -p      Send updates to standard output (avoids stickiness).  
12     -k kopt Use RCS kopt -k option on checkout. (is sticky)  
13     -r rev  Update using specified revision/tag (is sticky).  
14     -D date Set date to update from (is sticky).  
15     -j rev  Merge in changes made between current revision and rev.
```



```

16      -I ign  More files to ignore (! to reset).
17      -W spec Wrappers specification line.
18 (Specify the --help global option for a list of other help options)

```

Vamos admitir que o usuário `_2` criou e adicionou o arquivo `usuario2.txt`, realizando as tarefas a seguir:

Exemplo:

emacs usuario2.txt

cvs add usuario2.txt

cvs add: scheduling file 'usuario2.txt' for addition

cvs add: use 'cvs commit' to add this file permanently

cvs commit -m "arquivo adicionado pelo usuario_2"

cvs commit: Examining .

RCS file: /home/REPOSITORIO/exemplo-biblioteca-gnu/usuario2.txt,v done

Checking in usuario2.txt;

/home/REPOSITORIO/exemplo-biblioteca-gnu/usuario2.txt,v <-- usuario2.txt

initial revision: 1.1

done

Veja a seguir como deixar o diretório de trabalho do usuário_1 com os arquivos atualizados:

```
cd /tmp/usuario_1/exemplo-biblioteca-gnu  
cvs update
```

```
cvs update: Updating .
```

```
U leiam.txt
```

```
U usuario2.txt
```

Observe que o arquivo `usuario2.txt`, criado pelo usuário_2, foi adicionado ao diretório de trabalho do usuário_1. Como o usuário_2 também modificou o arquivo `leiam.txt`, o *cvs* também atualizou o arquivo `leiam.txt` (misturando-o com o `leiam.txt` do diretório de trabalho).

Quando ocorre uma mistura entre um arquivo do diretório de trabalho com um arquivo do repositório, o *cvs* cria no diretório de trabalho uma cópia oculta do arquivo original (`.nomeDoArquivo.versão`).

1.8.2 Como verificar diferenças entre arquivos

O programa *cvs* tem suporte interno ao programa *diff* (veja Capítulo ?? - *Os programas diff, patch, indent*), permitindo comparar os arquivos que estão sendo utilizados no diretório de trabalho com os do repositório.

Protótipo:

```
# Compara o arquivo do diretório de trabalho com o arquivo do repositório
cvs diff arquivo
# Verifica diferenças de todos os arquivos
cvs diff
# Verifica diferenças entre o último commit e o staged area
git diff --cached
```

Suponha que o usuário `_2` modificou o arquivo `leiamе.txt`, acrescentando o texto “Nova alteração realizada pelo usuário `_2`”. Depois enviou o arquivo `leiamе.txt` para o repositório usando um `cvs commit -m “nova atualização no leiamе.txt”`. Agora, o usuário `_1` está com o arquivo `leiamе.txt` numa versão antiga (desatualizado). O usuário `_1` pode verificar as diferenças entre seus arquivos e os do repositório usando o comando `cvs diff`. Veja sequência na listagem 1.4.

Listing 1.4: Saída do comando: ***cvs-diff***

```
1 # Usuário_2
2 [andre@suporte3 lib-gnu]$ emacs leiamе.txt
3 [andre@suporte3 lib-gnu]$ cvs ci -m "nova atualização no leiamе"
4 cvs commit: Examining .
5 Checking in leiamе.txt;
6 /home/REPOSITOARIO/exemplo-biblioteca-gnu/leiamе.txt,v <-- leiamе.txt
7 new revision: 1.5; previous revision: 1.4
8 done
```

```

9
10 # Usuário_1
11 [andre@mercurio exemplo-biblioteca-gnu]$ cvs diff
12 cvs diff: Diffing .
13 Index: leiame.txt
14 =====
15 RCS file: /home/REPOSITORY/exemplo-biblioteca-gnu/leiame.txt,v
16 retrieving revision 1.5
17 diff -r1.5 leiame.txt
18 7,11d6
19 < Nova alteração realizada pelo usuário_2
20 cvs diff: Diffing .libs
21 cvs diff: Diffing novoDir

```

Note o que falamos lá no início, o cvs não tem mecanismos de troca de mensagens, então, pode ocorrer de dois usuários modificarem o mesmo arquivo. Para evitar isso o ideal é criar ramos de trabalho e utilizar alguma ferramenta de comunicação para evitar as duplicações.

1.9 Desenvolvimento colaborativo *git*

A Figura 1.6 mostra uma sequência de uso do *cvs* envolvendo o desenvolvimento de um sistema por mais de um usuário.

O desenvolvimento colaborativo é o padrão hoje em dia no que se refere ao desenvolvimento de sistemas *de software* e sistemas de engenharia. Por exemplo, o *gnome* é desenvolvido por centenas de programadores espalhados por todas as partes do mundo. Um novo modelo de carro é desenvolvido por diversas equipes em diferentes locais do planeta. Isto só é possível com a disponibilização de ferramentas como o *cvs* e *git* para o compartilhamento dos arquivos de forma rápida e segura.

Mas como funciona na prática o desenvolvimento colaborativo?

- O coordenador do projeto (usuário_1) importa os arquivos iniciais do projeto para dentro do repositório, dando início ao projeto (veja Figura 1.6).
- A seguir, comunica as pessoas sobre o desenvolvimento de um novo projeto.
- Os interessados enviam mensagens de e-mail para o coordenador, informando o interesse em participar do projeto e dando informações que comprovem que tem capacidade de colaborar (seja no desenvolvimento de código, seja na documentação dos sistemas).
- Os habilitados são cadastrados no *cvs* (veja seção ??) e ou recebem informações para acesso ao repositório.

- Agora, os usuários cadastrados (usuário_1, usuário_2, usuário_3) podem acessar os arquivos do repositório com um comando *cvs checkout* (criando em suas máquinas um diretório de trabalho que é uma cópia fiel do sistema armazenado no repositório). Observe na Figura 1.6, que mais de um usuário pode trabalhar com os arquivos ao mesmo tempo.
- No final do dia, após ter feito algumas alterações (ex: incluído novos arquivos), o usuário_1 envia os arquivos modificados para o repositório com um comando *cvs commit*.
- Logo após, o usuário_2 faz o mesmo, mas como existem arquivos atualizados no repositório, o *cvs* informa o usuário_2 que antes de enviar os arquivos para o repositório, ele deve atualizar os arquivos do diretório de trabalho. Depois de dar um comando *cvs update* o usuário_2 vai poder enviar os arquivos com um comando *cvs commit*.
- Observe que no outro dia pela manhã, todos os usuários antes de começar a trabalhar, executam um comando *cvs update* (para manter os arquivos atualizados), e no final do dia os comandos *cvs update* e *cvs commit*.
- Este ciclo se repete todos os dias, até que um determinado *release* seja finalizado (veja seção ??). Opcionalmente, o chefe do projeto pode criar ramos paralelos (veja seção ?? e Figura ??).

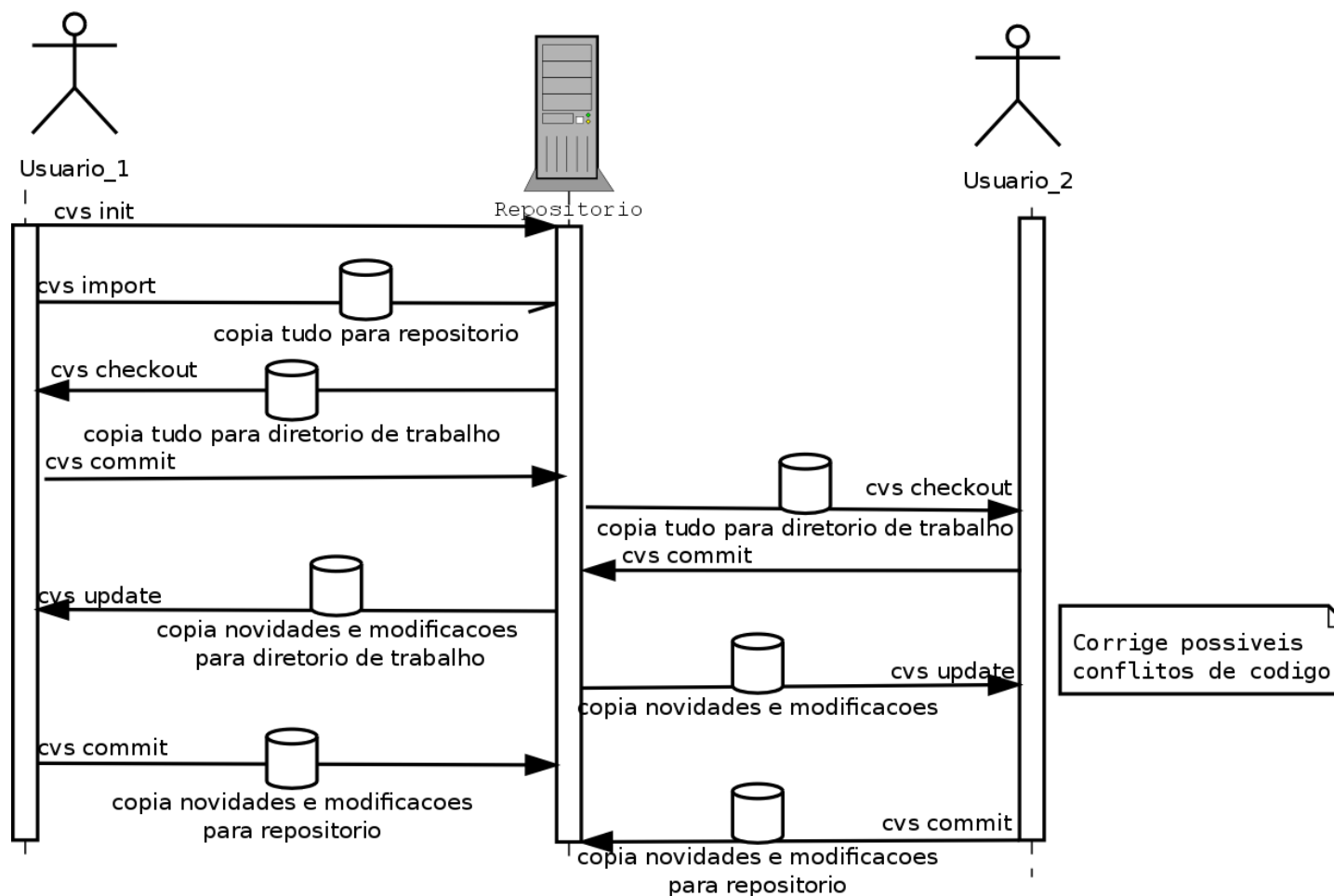


Figura 1.6: *cvs*: Sequência de trabalho para desenvolvimento colaborativo

1.10 *Tag's e releases*

Descrevemos no início deste capítulo o que é um *release* e um *tag* (seção ??). Apresentamos a seguir como criar e usar *releases* e *tags*.

1.10.1 Como criar *tag's* - versões de marcação

Como visto na seção ??, cada arquivo do repositório terá um número de versão. Entretanto, você pode realizar diversas modificações no arquivo `leiametext` (gerando as versões 1.1 -> 1.2 -> 1.3 -> 1.4), algumas modificações no arquivo `CPonto.h` (versões 1.1 -> 1.2 -> 1.3) e duas modificações no arquivo `CPonto.cpp` (1.1->1.2). Ou seja, cada arquivo tem um número de versão diferente. Seria interessante se você pudesse se referir a todos os arquivos do projeto em uma determinada data com um mesmo nome simbólico. Um *tag* é exatamente isto, um nome simbólico dado ao projeto após terminada determinada etapa do projeto.

Assim, quando uma etapa do projeto é concluída, pode-se criar um nome simbólico que será utilizado para recuperar todos os arquivos do projeto nessa etapa. Observe na Figura 1.7 que a versão de cada arquivo não é alterada.

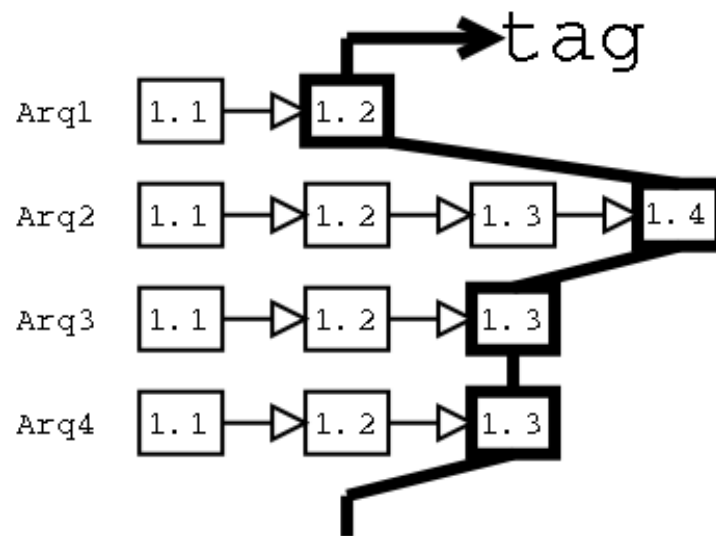


Figura 1.7: *cvs*: Como funciona um *tag*

Protótipo

Para criar um tag para um único arquivo:

```
cd diretório_de_trabalho
```

```
cvs tag nome_simbólico nome_arquivo
```

Para criar um tag para todos os arquivos do projeto:

```
cvs tag nome_simbólico
# Para recuperar arquivos de um tag:
cvs checkout -r nome_simbólico nome_projeto.
# Um tag pode ser eliminado:
cvs rtag -d nome_release_simbólico
```

Veja na listagem 1.5 a saída do comando *cvs tag Etapa_1* executada no diretório de trabalho do usuário_1. Observe que o número da versão dos arquivos não é modificada.

Listing 1.5: Saída do comando: ***cvs tag Etapa-1***

```
1[andre@suporte3 exemplo-biblioteca-gnu]$ cvs tag Etapa_1
2cvs tag: Tagging .
3T CPonto.h
4T CPonto.cpp
5T ProgCPonto.cpp
6T leiname.txt
7T usuario2.txt
8cvs tag: Tagging CVSR00T
9T CVSR00T/modules
```

Vamos criar um diretório usuário_3, que é acessado pelo usuário 3, o mesmo pode recuperar a versão completa do projeto, utilizando o *tag* que acabamos de criar. Observe que para obter uma cópia do módulo lib-gnu, utilizamos *cvs*

checkout lib-gnu, e para obter uma cópia do *tag* *Etapa_1* do módulo *lib-gnu*, utilizamos *cvs checkout -r Etapa_1 lib-gnu*. Ou seja, apenas adicionamos após o comando *checkout*, o parâmetro *-r* e o nome do *tag*.

```
mkdir /tmp/usuario_3  
cd /tmp/usuario_3  
cvs checkout -r Etapa_1 lib-gnu
```

```
cvs checkout: Updating lib-gnu  
U lib-gnu/CPonto.h  
U lib-gnu/CPonto.cpp  
U lib-gnu/ProgCPonto.cpp  
U lib-gnu/leiametext  
U lib-gnu/usuario2.txt
```

1.10.2 Como criar *release*'s (versões de distribuição)

Geralmente utilizamos um *tag* para criar uma versão do projeto que esteja funcionando ou que compreenda a finalização de um determinado conjunto de tarefas que estavam pendentes. Assim, com o nome do *tag* você pode recuperar o projeto em uma determinada etapa utilizando um nome abreviado.

Depois de finalizada uma versão do programa você pode criar um *release* (pacote funcional). A diferença entre o *tag* e o *release* é que o *tag* não modifica a versão dos arquivos do projeto e o *release*, sim, dando a todos os arquivos um mesmo número de versão. Veja na Figura 1.8 como fica um novo *release*, observe que com a aplicação do *release*, todos os arquivos passaram a ter o mesmo número de versão.

- Um *release* é geralmente um pacote funcional e se aplica a todos os arquivos do projeto.
- Depois de definido o *release*, este não pode ser modificado.
- Você deve criar um *release* sempre que tiver finalizado uma parte importante de seu programa.

Veja a seguir o protótipo para criar um *release*.

Protótipo

Para criar um release:

```
cvs commit -r número_release
```

Para criar um release e já deletar a cópia do diretório local:

```
cvs release -d diretório_de_trabalho
```

No exemplo a seguir o `usuario_2` atualiza seus arquivos e a seguir cria o release 2.

Exemplo:

```
cd /tmp/usuario_2/lib-gnu  
cvs update
```

```
cvs update: Updating .
```

```
U leiname.txt
```

```
cvs update: Updating CVSROOT
```

```
cvs commit -r 2 -m “finalizada versão 2.0”
```

Veja na listagem [1.6](#) a saída do comando `cvs commit -r 2`. Observe que todos os arquivos passam a ter o mesmo número de versão (2.1). Veja Figura [1.8](#).

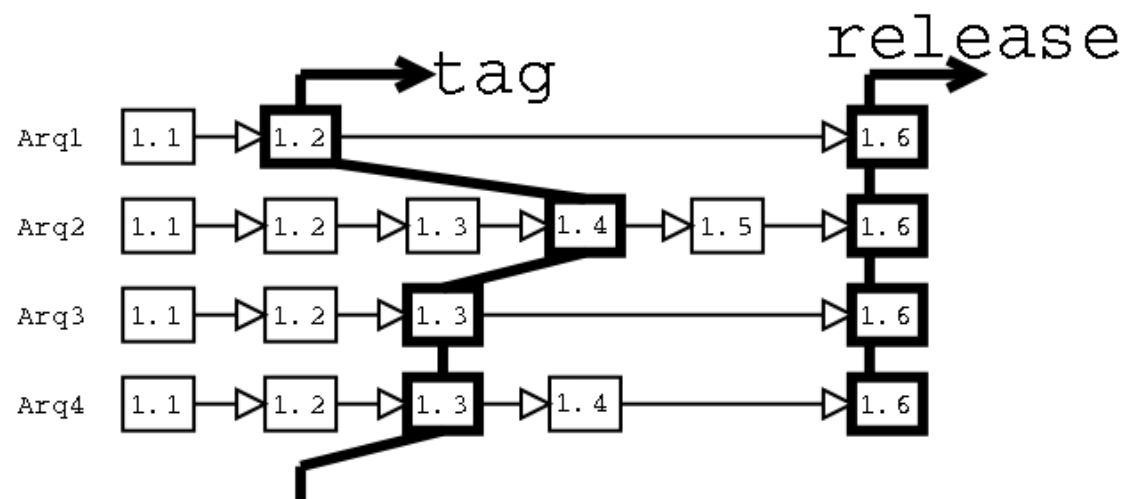


Figura 1.8: *cv*s: Como funciona um *release*

Listing 1.6: Saída do comando: *cv*s *commit -r 2*

```

1 [root@mercurio lib-gnu]# cvs commit -r 2
2 cvs commit: Examining .
3 cvs commit: Examining .libs
4 cvs commit: Examining novoDir
5 Checking in Makefile;
6 /home/REPOSITARIO/exemplo-biblioteca-gnu/Makefile,v <-- Makefile
    
```

```
7 new revision: 2.1; previous revision: 1.1
8 done
9 Checking in arquivo-usuario2.txt;
10 /home/REPOSITARIO/exemplo-biblioteca-gnu/arquivo-usuario2.txt,v  <--  arquivo-usuario2.txt
11 new revision: 2.1; previous revision: 1.1
12 done
13 ....
14 ....
15 Checking in leiname.txt;
16 /home/REPOSITARIO/exemplo-biblioteca-gnu/leiname.txt,v  <--  leiname.txt
17 new revision: 2.1; previous revision: 1.3
18 done
```

1.10.3 Como recuperar módulos e arquivos

O *cvs* permite que tanto as versões novas como as antigas possam ser recuperados. De uma maneira geral, basta passar o nome do arquivo e sua versão (*tag*, *release*). Também podemos baixar um *release* antigo, passando o nome do *release* e o nome do projeto no repositório.

Protótipo

Para recuperar um release:

```
cvs checkout -r nome_release nome_projeto
# Para recuperar um release usando nome do módulo
cvs checkout -r nome_release nome_módulo.
# Para recuperar um arquivo de uma versão antiga:
cvs update -p -r release arquivo > arquivo
```

Argumento Descrição

-p Envia atualizações para saída-padrão (a tela).

-r release Indica a seguir o nome do release.

arquivo O nome do arquivo a ser recuperado.

> arquivo Redireciona da tela para o arquivo arquivo.

No exemplo a seguir, recupera-se o arquivo `leiametext` do *tag* `Etapa_1`. A opção `-p` redireciona a saída para a tela, e `>` redireciona para o arquivo `leiametext-Etapa_1.txt`.

```
cd usuário_3
cvs update -p -r Etapa_1 leiametext > leiametext-Etapa_1.txt
```

Checking out leiametext

RCS: /home/REPOSITORIO/exemplo-biblioteca-gnu/leiametext,v VERS: 1.5

1.11 Ramos e misturas (*branching and merging*)

O programa *cvs* permite que você trabalhe com ramos de desenvolvimento de seu projeto. Por exemplo, você pode criar um ramo principal e ramos derivados. Posteriormente, você poderá misturar os diferentes ramos.

Depois de finalizado um *release* de um programa é bastante usual a criação de três ramos; o ramo principal, o ramo de atualizações (*patch*) e o ramo de desenvolvimento da nova versão. Veja na Figura 1.9 a disposição dos ramos de um projeto. Observe que se o número da versão de um arquivo do ramo principal é 1.2, de um arquivo de um ramo secundário é 1.2.2.1, 1.2.2.2.

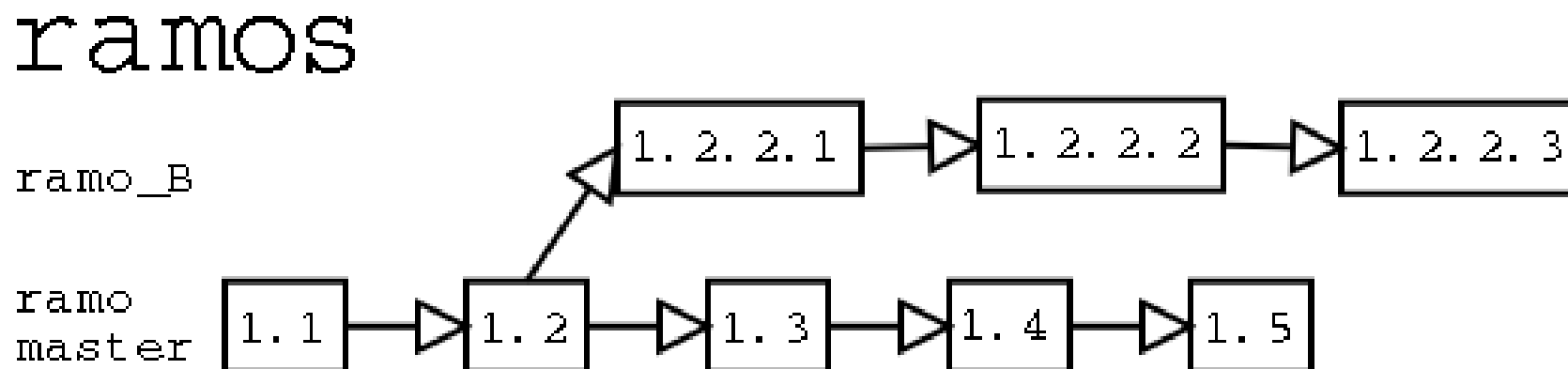


Figura 1.9: *cvs*: Como ficam os ramos

1.11.1 Como trabalhar com ramos

Veja a seguir o protótipo para trabalhar com ramos.

Protótipo:

```
# Para criar um ramo no diretório de trabalho (-b de branch):
git branch nome_do_amo

# Para criar um ramo a partir de um release existente:
cvs rtag -b -r nome_do_release nome_do_amo path_no_repositório

# Obtendo uma cópia do ramo:
cvs checkout -r nome_do_amo nome_projeto

# Atualização dos arquivos locais de um dado ramo:
cvs update -r nome_do_amo nome_projeto
cvs update -r nome_do_amo nome_modulo

# Para obter lista dos branches
git branch

# Para alternar para outro ramo
```

```
git switch nomeRamo
# Para mesclar dois ramos
git switch ramo1
git merge ramo2
# Para deletar um ramo
git branch -d nomeRamo
# Para deletar de forma permanente um ramo
git branch -D nomeRamo.
```

Observe que rtag é uma abreviatura para *repository tag*.

Para saber com qual ramo você está trabalhando, verifique o nome do ramo na seção "Existing tags", listada pelo comando *status -v*:

```
cvs status -v nome_arquivo
```

Digamos que você esteja trabalhando no projeto e que o *release* 2.0 já foi suficientemente testado, podendo ser publicado. Então, você cria o *release* 2.0.

```
# Ramo principal: exemplo-biblioteca-gnu
# Para criar um release usamos
cvs commit -r 2 -m “finalizada versão 2.0”
```

Agora você pode criar um ramo de atualização da versão 2.0 (chamado patch), que conterá os arquivos da versão 2.0, mas com correções de *bugs* que tenham sido localizados. Assim, se foi identificado algum bug na versão 2.0, você faz as alterações no ramo lib-gnu-patch, deixando o *release* 2.0 inalterado:

```
# Para criar um ramo de atualização
cd usuario_1
cvs tag -b lib-gnu-patch

cvs tag: Tagging .
T CPonto.h
T CPonto.cpp
T ProgCPonto.cpp
T leiametext
T usuario2.txt
```

Finalmente, você pode criar um ramo de desenvolvimento (instável), onde ficarão os arquivos da nova versão (observe o número do novo ramo):

```
# Ramo: lib-gnu-desenvolvimento
cvs tag -b lib-gnu-desenvolvimento
```

cvs tag: Tagging .

T CPonto.h

T CPonto.cpp

T ProgCPonto.cpp

T leiname.txt

T usuario2.txt

Ou seja, vamos ter três ramos: o *release* 2.0, que não será mais alterado. O patch, que vai ter as correções de bugs da versão 2.0, e o 2.2 que terá os arquivos da nova geração do projeto.

1.11.2 Como mesclar duas versões de um arquivo

Com a opção -j, você pode verificar as diferenças entre duas versões de um arquivo. Veja a seguir o protótipo.

Protótipo:

cvs update -j versãoVelha -j versãoNova nomeArquivo

Veja a seguir um exemplo. Observe a mensagem apresentada. O *cvs* recupera a versão 2.1 e a versão 1.1 e mistura as duas no arquivo leiname.txt.

Exemplo:

```
cd usuario_3  
cvs update -j 1.1 -j 2.1 leiame.txt
```

U leiame.txt

RCS file: /home/REPOSITORIO/exemplo-biblioteca-gnu/leiame.txt,v

retrieving revision 1.1

retrieving revision 2.1

Merging differences between 1.1 and 2.1 into leiame.txt.

1.11.3 Como mesclar o ramo de trabalho com o ramo principal

Digamos que o usuário_2 baixou o ramo *lib-gnu-patch* e fez alterações no arquivo ProgCPonto.cpp (ex: corrigiu um *bug*). Depois enviou tudo para o repositório com um *commit*.

```
cvs co -r lib-gnu-patch lib-gnu
```

cvs checkout: Updating lib-gnu

U lib-gnu/CPonto.h

U lib-gnu/CPonto.cpp

U lib-gnu/ProgCPonto.cpp

U lib-gnu/leiametext

U lib-gnu/usuario2.txt

emacs ProgCPonto.cpp

cvs ci -m “atualizado ProgCPonto”

cvs commit: Examining .

Checking in ProgCPonto.cpp;

/home/REPOSITORIO/exemplo-biblioteca-gnu/ProgCPonto.cpp,v

<-- ProgCPonto.cpp

new revision: 2.1.2.1; previous revision: 2.1

done

Agora você quer incluir as alterações do ramo gnome-patch no ramo principal. Veja a seguir um roteiro para mesclar os dois ramos.

1. Obtém uma cópia do ramo principal:

cd usuario_5

```
cvs checkout lib-gnu
cd lib-gnu
```

2. Pede para o *cvs* adicionar ao diretório de trabalho os arquivos do ramo *gnome-patch*, utilizando o comando *cvs update* com a opção *-j*.

```
cvs update -j lib-gnu-patch
```

```
cvs update: Updating .
```

```
RCS file:
```

```
/home/REPOSITORIO/exemplo-biblioteca-gnu/ProgCPonto.cpp,v
```

```
retrieving revision 2.1 retrieving
```

```
revision 2.1.2.1
```

```
Merging differences between 2.1 and 2.1.2.1 into
```

```
ProgCPonto.cpp
```

3. Resolve os possíveis conflitos. Alguns arquivos que tenham sido modificados por outros usuários podem ter conflitos de código; você precisa resolvê-los.

```
# ...correção de possíveis conflitos de código...
```


4. Envia os arquivos de volta para o repositório, atualizando o repositório. Agora os arquivos do ramo estão atualizados.

cvs commit -m “Ramo mestre mesclado com ramo lib-gnu-patch”

```
cvs commit: Examining .  
Checking in ProgCPonto.cpp;  
/home/REPOSITORIO/exemplo-biblioteca-gnu/ProgCPonto.cpp,v  
<-- ProgCPonto.cpp  
new revision: 2.2; previous revision: 2.1  
done
```

Veja na listagem 1.7, um exemplo.

Listing 1.7: Exemplo: Mesclando dois ramos

```
1 # Baixa o ramo principal  
2 [andre@suporte3 Usuario_4]$ cvs co lib-gnu  
3 cvs checkout: Updating lib-gnu  
4 U lib-gnu/CPonto.h  
5 U lib-gnu/CPonto.cpp  
6 U lib-gnu/ProgCPonto.cpp  
7 U lib-gnu/leiametext  
8 U lib-gnu/usuario2.txt
```

```
9
10 [andre@suporte3 Usuario_4]$ cd lib-gnu/
11 # Copia os arquivos do ramo de patch para o diretório de trabalho
12 # Todos os arquivos são misturados
13 [andre@suporte3 lib-gnu]$ cvs update -j lib-gnu-patch
14 cvs update: Updating .
15 RCS file: /home/REPOSITARIO/exemplo-biblioteca-gnu/CPonto.h,v
16 retrieving revision 1.1
17 retrieving revision 1.1.1.1
18 Merging differences between 1.1 and 1.1.1.1 into CPonto.h
19 CPonto.h already contains the differences between 1.1 and 1.1.1.1
20 RCS file: /home/REPOSITARIO/exemplo-biblioteca-gnu/CPonto.cpp,v
21 retrieving revision 1.1
22 retrieving revision 1.1.1.1
23 Merging differences between 1.1 and 1.1.1.1 into CPonto.cpp
24 CPonto.cpp already contains the differences between 1.1 and 1.1.1.1
25 RCS file: /home/REPOSITARIO/exemplo-biblioteca-gnu/ProgCPonto.cpp,v
26 retrieving revision 1.1
27 retrieving revision 1.1.1.1
28 Merging differences between 1.1 and 1.1.1.1 into ProgCPonto.cpp
29 ProgCPonto.cpp already contains the differences between 1.1 and 1.1.1.1
30
31 # Faz as correções necessárias, testa o sistema e então dá um commit
32 # enviando para o repositório
```

```
33 [andre@suporte3 lib-gnu]$ cvs commit -m "Ramo mestre mesclado com lib-gnu-patch"
34 cvs commit: Examining .
35 Checking in leiname.txt;
36 /home/REPOSITARIO/exemplo-biblioteca-gnu/leiname.txt,v <-- leiname.txt
37 new revision: 2.2; previous revision: 2.1
38 done
```

1.12 Como verificar o estado do repositório

O programa *git* tem um conjunto de comandos que você pode utilizar para verificar o estado dos arquivos armazenados no repositório.

1.12.1 Como verificar o histórico das alterações

Você pode obter uma lista com o histórico das alterações realizadas no repositório, veja o protótipo. O comando *cvs history* mostra: data, hora e nome do usuário, além do nome do projeto e do diretório de trabalho.

Protótipo:

cvs history

Veja na listagem 1.8 a saída do comando *cvs history*.

Listing 1.8: Saída do comando: *cvs history*

```
1[andre@suporte3 lib-gnu]$ cvs history
20 2012-06-01 20:49 +0000 andre CVSR00T          =CVSR00T=          /tmp/Usuario_1/exemplo-biblioteca-gnu/*
30 2012-06-01 20:47 +0000 andre exemplo-biblioteca-gnu =exemplo-biblioteca-gnu= /tmp/Usuario_1/*
40 2012-06-01 21:37 +0000 andre exemplo-biblioteca-gnu =lib-gnu=          /tmp/Usuario_5/*
```

1.12.2 Como verificar as mensagens de *log* (*loginfo*)

Você pode obter uma lista com as mensagens enviadas com o comando *cvs commit*. O comando *cvs log* mostra o caminho do arquivo no repositório, o nome do diretório de trabalho, sua versão, nomes simbólicos, revisões (totais e selecionadas), e para cada anotação realizada (o número da versão, quem fez a atualização, quando, o número de linhas modificadas e um comentário).

Protótipo:

cvs log arquivo

Veja na listagem 1.9 a saída do comando *cvs log leiname.txt*. Observe as diferentes revisões e anotações, o nome do autor. Observe nos nomes simbólicos que o *tag* Etapa_1 é uma referência à versão 1.5. Isto significa que quando o usuário solicitar o Etapa_1, este corresponde à versão 1.5 do arquivo leiname.txt.

Listing 1.9: Saída do comando: *cvs log leiname.txt*

```

1[andre@suporte3 lib-gnu]$ cvs log leiname.txt
2RCS file: /home/REPOSITARIO/exemplo-biblioteca-gnu/leiname.txt,v
3Working file: leiname.txt
4head: 2.1
5branch:
6locks: strict
7access list:
8symbolic names:
9    lib-gnu-desenvolvimento: 2.1.0.4
10    lib-gnu-patch: 2.1.0.2
11    Etapa_1: 1.5
12keyword substitution: kv
13total revisions: 7;      selected revisions: 7
14description:
15-----
16revision 2.1
17date: 2012/06/01 21:22:43;  author: andre;  state: Exp;  lines: +0 -0
18Criado release 2 pelo usuário_2
19-----
20revision 1.6
21date: 2012/06/01 21:20:15;  author: andre;  state: Exp;  lines: +1 -0
22modificação do usuário 3
23-----
24revision 1.5

```

```

25 date: 2012/06/01 21:10:43;  author: andre;  state: Exp;  lines: +1 -0
26 atualizado arquivo leiametext
27 -----
28 revision 1.4
29 date: 2012/06/01 21:02:32;  author: andre;  state: Exp;  lines: +1 -0
30 atualização
31 -----
32 revision 1.3
33 date: 2012/06/01 20:56:01;  author: andre;  state: Exp;  lines: +0 -0
34 recuperei leiametext
35 -----
36 revision 1.2
37 date: 2012/06/01 20:55:18;  author: andre;  state: dead;  lines: +0 -0
38 removido leiametext
39 -----
40 revision 1.1
41 date: 2012/06/01 20:54:25;  author: andre;  state: Exp;
42 adicionado arquivo leiametext
43 =====

```

O comando log também aceita intervalos de datas, como em:

```

cvs log -d "05/5/2012"
cvs log -d ">05/6/2012"

```

cvs log -d “>=05/12/2006 ; <=05/01/2012”

1.12.3 Como verificar as anotações

Você pode obter uma lista das anotações realizadas. O comando *cvs annotate* mostra a versão, o nome do usuário, a data em que as alterações foram realizadas e uma mensagem de log.

Protótipo:

cvs annotate leiname.txt

Listing 1.10: Saída do comando: *cvs annotate leiname.txt*

```
1 [andre@suporte3 lib-gnu]$ cvs annotate leiname.txt
2
3 Annotations for leiname.txt
4 *****
5 1.1      (andre    01-Jun-12): ...inclua informações sobre o projeto... (usuário_1)
6 1.4      (andre    01-Jun-12): ...alterações feitas pelo usuário 2...
7 1.5      (andre    01-Jun-12): ...Nova alteração realizada pelo usuário_2...
8 1.6      (andre    01-Jun-12): ...Nova alteração realizada pelo usuário_3...
```

1.12.4 Como verificar o *status* dos arquivos

O comando *status* mostra uma série de informações a respeito dos arquivos do diretório de trabalho. Dentre as informações apresentadas pode-se citar: nome do arquivo, versão local e do repositório, e os atributos persistentes. Veja a seguir o protótipo e as informações listadas pelo comando *status*; veja as informações retornadas na Tabela [1.2](#).

Protótipo:

cvs status

<i>Parâmetro</i>	<i>Descrição</i>
------------------	------------------

<i>-v</i>	<i>Mostra adicionalmente os tag's e releases do arquivo.</i>
-----------	--

<i>-R</i>	<i>Processamento recursivo.</i>
-----------	---------------------------------

<i>-l</i>	<i>Somente este diretório (não recursivo).</i>
-----------	--

Tabela 1.2: Informações listadas pelo comando *cvs status*

Informação	Descrição
<i>Up-to-date</i>	O arquivo não foi alterado (é o mesmo do repositório).
<i>Locally modified</i>	O arquivo foi modificado localmente.
<i>Locally added</i>	O arquivo foi adicionado localmente.
<i>Locally removed</i>	O arquivo foi removido localmente.
<i>Needs checkout, patch</i>	O arquivo disponibilizado no repositório é uma versão mais nova (foi alterado por terceiros) e precisa ser atualizado. Com um comando <i>update</i> , copia o arquivo do repositório para o local de trabalho, mesclando-o com o arquivo já existente no local de trabalho.
<i>File had conflicts on merge</i>	O arquivo apresenta conflitos após a mistura.
<i>Unknown</i>	O arquivo local é desconhecido (ainda não foi adicionado ao repositório).

Veja na listagem 1.11 a saída do comando *cvs status* executado sobre o arquivo *leiam.txt* pelo usuário_1. Observe

que o arquivo foi localmente modificado.

Listing 1.11: Saída do comando: *cvs status -v leiname.txt*

```

1 [andre@suporte3 lib-gnu]$ cvs status -v leiname.txt
2 =====
3 File: leiname.txt          Status: Up-to-date
4
5 Working revision:         2.1      Wed Jun  1 21:22:43 2012
6 Repository revision:     2.1      /home/REPOSITORIO/exemplo-biblioteca-gnu/leiname.txt,v
7 Sticky Tag:              lib-gnu-patch (branch: 2.1.2)
8 Sticky Date:             (none)
9 Sticky Options:          (none)
10
11 Existing Tags:
12     lib-gnu-desenvolvimento      (branch: 2.1.4)
13     lib-gnu-patch                (branch: 2.1.2)
14     Etapa_1                     (revision: 1.5)

```

1.13 Alguns exemplos de uso com vários usuários - Desenvolvimento em grupo

Até agora vimos o funcionamento do *git* com um único usuário. Ele é muito parecido com o *cvs*; a diferença é que o repositório fica armazenado dentro do diretório de trabalho (subdiretório *.git*). Agora vamos ver o desenvolvimento colaborativo - em grupo.

Um dos aspectos relacionados ao desenvolvimento de software de forma profissional é que o pacote de software costuma ter mais de uma versão:

- **main:** é o ramo principal.
- **Desenvolvimento:** na versão de desenvolvimento ficam os arquivos novos, aquilo que esta sendo desenvolvido. Teremos aqui marcadores/tags para *features* concluídas. Quando uma versão estiver finalizada (devidamente testada e documentada) teremos um release.
- **Fi-Nome:** é um ramo temporário usado para implementar/testar/documentar uma determinada característica. Composta pelas linhas.

Fi-d: Linha de desenvolvimento da Fi.

Fi-t: Linha de teste da Fi.

Fi-w: Linha de documentação da Fi.

- **Teste:** versão usada pela equipe responsável pela procura de *bugs* e validação dos sistemas. Também conhecida como alfa/beta.
- **Produção:** é a versão normalmente distribuída. Esta versão já foi devidamente testada e é usada pelos usuários “comuns”.

Cada versão costuma ter uma identificação do tipo:

- Versão.subversão.subsubversão
 - **Versão:** mudanças de paradigma, grandes mudanças.
 - **Subversão:** adição de novas funcionalidades/*features*, mudanças de partes do sistema.
 - **Subsubversão:** correção de bugs.
- Ex: kernel-3.3.7-1.fc17.x86_64

Adicionalmente, o desenvolvimento requer o constante compartilhamento e sincronização de arquivos pela equipe de desenvolvimento. Veremos a seguir exemplos de comandos do *git* usados para troca de arquivos entre a equipe de desenvolvimento.

1. O gerenciamento das diferentes versões do sistema de software é feita através do uso de *branches*/ramos. No *git*, o ramo inicial é chamado de *master*. Para criar os ramos de Desenvolvimento, Teste e Produção, execute os seguintes comandos.

```
git checkout main
git branch Desenvolvimento
# opcional, pode-se usar o próprio master como ramo de Desenvolvimento
git checkout main
git branch Teste
git checkout main
git branch Producao
```

2. Para mudar para o ramo de Teste:

```
git checkout Teste
    Switched to branch 'Teste'
echo "Ramo de Teste: changeLog" >> changeLog
git add changeLog
git commit -m"Adicionado arquivo changeLog"
```

ao ramo de Teste"

```
[Teste c97d06b] Adicionado arquivo changeLog ao ramo de  
Teste 1 file changed, 1 insertion(+) create mode 100644 changeLog
```

git status

```
[bueno17@bueno-notebook workdir]$ git status
```

```
# On branch Teste
```

```
# Untracked files:
```

```
#   (use "git add <file>..." to include in what will be committed)
```

```
#
```

```
# .gitignore
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

git checkout Producao

```
git checkout Producao
```

```
echo "Ramo de Produção: changeLog" >> changeLog
```

```
git add changeLog
```

```
git commit -m"Adicionado arquivo changeLog ao ramo de Produção"
```

```
[Producao 2d96e6e] Adicionado arquivo changeLog ao ramo de Produção
```

```
1 file changed, 1 insertion(+)
create mode 100644 changeLog

git status

# On branch Producao
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
# .gitignore
nothing added to commit but untracked files present (use "git add" to track)
```

3. Como o uso de ramos/branches é simples, é normal criar ramos/branches para cada funcionalidade a ser desenvolvida.

```
# necessário voltar para ramo de desenvolvimento
git checkout Desenvolvimento

Switched to branch 'Desenvolvimento'

git branch DesenvolvimentoFuncionalidadeX
git checkout DesenvolvimentoFuncionalidadeX

git branch DesenvolvimentoFuncionalidadeX
```

```
echo "Ramo DesenvolvimentoFuncionalidadeX" >> changeLog
git add changeLog
git commit -m"Adicionado arquivo changeLog ao ramo DesenvolvimentoFuncionalidadeX"

[DesenvolvimentoFuncionalidadeX 96883a2] Adicionado arquivo changeLog
ao ramo DesenvolvimentoFuncionalidadeX
1 file changed, 1 insertion(+)
create mode 100644 changeLog

git merge Desenvolvimento
Already up-to-date.
```

Note que neste caso, após finalizar a funcionalidade X, é necessário a unificação do ramo DesenvolvimentoFuncionalidadeX com o ramo Desenvolvimento; o que é feito com o comando *git merge*. É importante ressaltar que a mistura de ramos de desenvolvimento pode implicar no aparecimento de códigos com conflito. Isto ocorre quando um mesmo arquivo foi modificado nos dois ramos, neste caso o desenvolvedor precisa definir qual código vai ficar.

4. Agora que já sabemos como criar diferentes ramos de código e como unificá-los, vamos ver como os arquivos podem ser enviados entre os diferentes membros da equipe de desenvolvimento. Os comandos utilizados são: “*git clone*”, “*git pull*” e o “*git push*”.

- *git clone*: cria um clone do repositório. Além dos arquivos de desenvolvimento contém o subdiretório `.git` (o repositório com todo histórico de desenvolvimento). Armazena referência ao repositório original(master).
 - *git pull*: copia do repositório master para o repositório de trabalho.
 - *git push*: copia do repositório de trabalho para o repositório master.
5. O repositório a ser compartilhado pode estar num servidor na internet e o acesso pode ser feito usando SSH, HTTP, HTTPS. Quando o repositório vai ser compartilhado a criação é feita usando as opções `--shared --bare`. A opção `--shared` indica que é um repositório a ser compartilhado. A opção `--bare`....

```
# Gerente do projeto
mkdir repositorioProjetoAcoplamentoPocoReservatorio
cd repositorioProjetoAcoplamentoPocoReservatorio
git init --shared --bare

[bueno17@bueno-notebook repositorio-compartilhado]$
git init --shared --bare
Initialized empty shared Git repository in /home/bueno17/Documentos
/2-Ensino/1-Apostilas-Livros-Pessoais/
0-Livro-CPP/LivroCPP-2nd/-01-lyx-imagens-listagens/listagens
/Cap-42/git/repositorio-compartilhado/
```

6. Usuário Carlos

```
# Usuário Carlos configura acesso ao servidor e então clona o
# repositório (cria dentro do diretório atual uma cópia
# do repositório com o nome Carlos--workdir)
git clone repositorio-compartilhado Carlos-workdir
    Cloning into 'Carlos-workdir'...
    warning: You appear to have cloned an empty repository.
    done.
```

7. Usuário Santos

```
# Usuário Santos configura acesso ao servidor e
# então clona o repositório
git clone repositorio-compartilhado Santos-workdir
    Cloning into 'Santos-workdir'...
    warning: You appear to have cloned an empty repository.
    done.
```

```
# Usuário Santos cria arquivo novo, adiciona ao repositório
# e então envia para servidor
cd Santos-workdir/
cat >> Tarefas.Santos.txt...ctrl+d.
git add Tarefas.Santos.txt
git status

# On branch master
#
# Initial commit
#
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
# new file:   Tarefas.Santos.txt
#

git commit -m"Santos adicionou arquivo Tarefas.Santos.txt"

[master (root-commit) c08748b] Santos adicionou arquivo Tarefas.Santos.txt
1 file changed, 3 insertions(+)
```

```
    create mode 100644 Tarefas.Santos.txt
git status
# On branch master
nothing to commit (working directory clean)
# git push EndereçoRepositórioCompartilhado=origin; nomeRamo
git push origin master
Counting objects: 3, done.
Writing objects: 100% (3/3), 293 bytes, done.
Total 3 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
To /home/bueno17/Documentos/2-Ensino/1-Apostilas-Livros-Pessoais/0-Livro-CPP
/LivroCPP-2nd/listagens/Cap-42/git/repositorio-compartilhado
* [new branch]      master -> master
```

8. Usuário Carlos

```
# Usuário Carlos
# git pull EndereçoRepositórioCompartilhado=origin; nomeRamo
```

```
cd Carlos-workdir
git status
    # On branch master
    #
    # Initial commit
    #
    nothing to commit (create/copy files and use "git add" to track)
cat >> Tarefas.Carlos.txt...ctrl+d
git add .
git commit -m"Carlos adicionou arquivo
Tarefas.Carlos.txt"
    [master (root-commit) b236661] Carlos adicionou arquivo
    Tarefas.Carlos.txt
    1 file changed, 2 insertions(+)
    create mode 100644 Tarefas.Carlos.txt
git pull origin master
## Primeiro abre o editor de texto padrão (no meu caso o emacs),
## com o seguinte conteúdo
```

```

Merge branch 'master' of /home/bueno17/Documentos
/2-Ensino/1-Apostilas-Livros-Pessoais/
0-Livro-CPP/LivroCPP-2nd/listagens/Cap-42
/git/repositorio-compartilhado
# Please enter commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
Adicionado arquivo de tarefas do Carlos.
.
## A seguir aparece no terminal as mensagens
warning: no common commits
remote: Counting objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From /home/bueno17/Documentos/2-Ensino/1-Apostilas-Livros-Pessoais
/0-Livro-CPP/LivroCPP-2nd/listagens

```

```

/Cap-42/git/repositorio-compartilhado
* branch          master      -> FETCH_HEAD
Merge made by the 'recursive' strategy.
Tarefas.Santos.txt |    3 +++
1 file changed, 3 insertions(+)
create mode 100644 Tarefas.Santos.txt

git status

# On branch master
nothing to commit (working directory clean)

```

9. Note que podemos ter diferentes opções de desenvolvimento:

- (a) Centralizado: todas as modificações passam pelo repositório centralizado. Neste caso os usuários não trocam arquivos do projeto entre si, todas as trocas são feitas através do repositório.
- (b) Centralizado compartilhado: o sistema é desenvolvido por diferentes grupos de trabalho. Neste caso as modificações parciais são trocadas entre usuários de cada grupo, e, posteriormente são integradas através do repositório compartilhado.
- (c) Centralizado com gerente integração: neste caso cada desenvolvedor tem dois repositórios, um público e um privado. Os comandos *pull* são realizados em direção ao repositório privado; quando parte do trabalho foi

finalizada e testada, os arquivos são enviados para o repositório público com o comando *push*. O gerente de integração acessa, através do comando *pull*, cada repositório público, e faz a integração do sistema.

- (d) Finalmente, pode-se ter um sistema ainda mais avançado; Neste caso um(ou mais) usuário(s) validador(es) valida(m) o código público de cada desenvolvedor; somente depois da validação é que o gerente de integração unifica o sistema.

Acredito ser necessário o usuário de validação quando o código foi desenvolvido por novos desenvolvedores, ou quando o código envolva conceitos físicos/matemáticos elaborados, que precisam ser testados e validados por especialistas nestas questões. Em muitos casos o código é desenvolvido por um especialista em programação, sendo necessário o acompanhamento de um engenheiro, especialista nos conceitos físicos desenvolvidos.

1.14 Um diagrama com os principais comandos do *git*

Veja na Figura 1.10 um diagrama com os principais comandos do *git*.

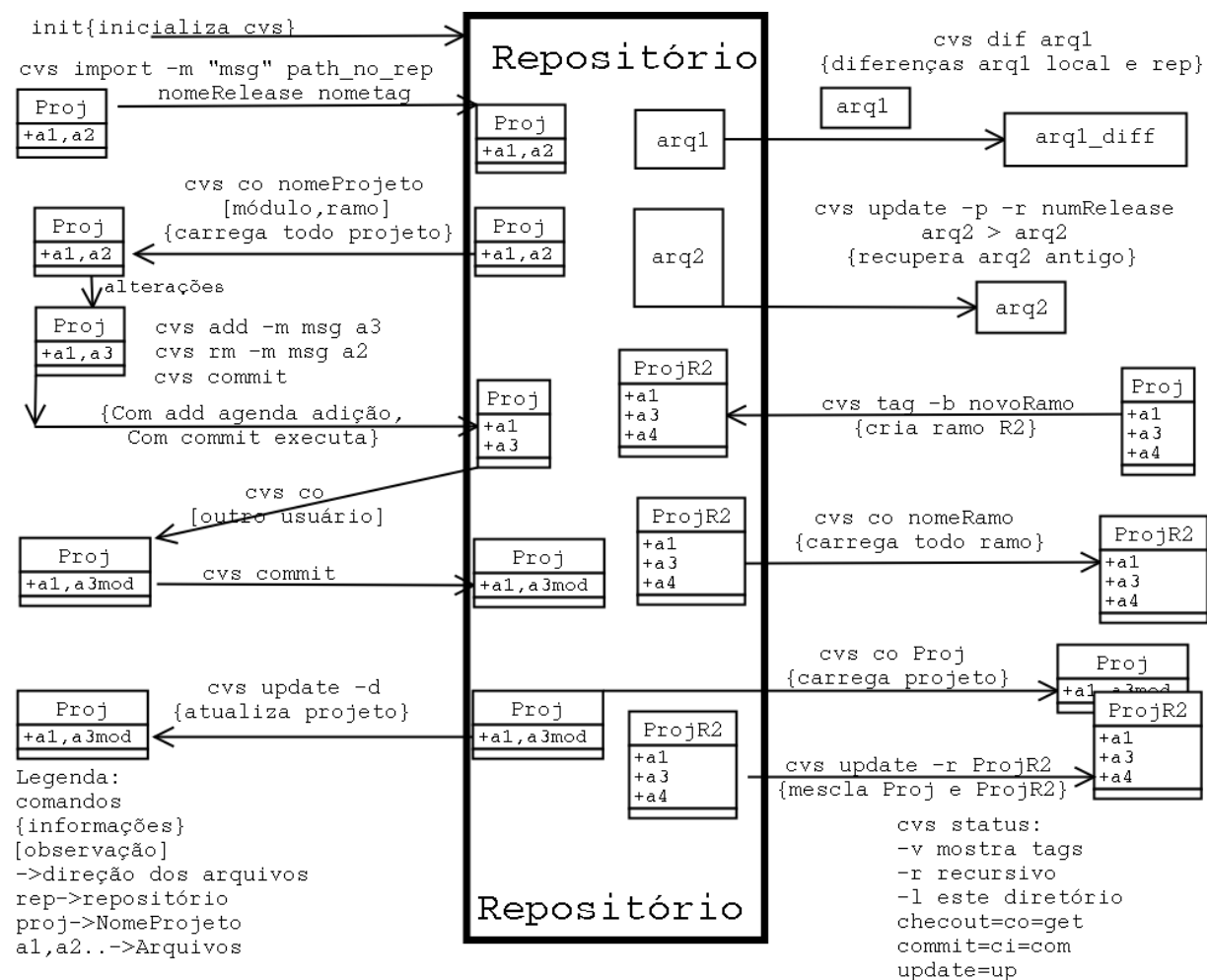


Figura 1.10: cvs: Diagrama com os comandos do *git*

1.15 Definindo uma estratégia para trabalho em grupo

Como o git é uma ferramenta de trabalho utilizada por um número muito grande de usuários e empresas, não é possível definir uma metodologia de trabalho (fluxo de trabalho) único. Nestes casos existem algumas sugestões e cabe a equipe que vai desenvolver o projeto definir quais procedimentos irá adotar.

Pontos a considerar na hora de definir um procedimento de uso do *git* em equipes:

- A equipe já conhece o *git* e tem experiência.
 - Se a equipe tem pouca experiência o procedimento de uso a ser adotado deve ser simples e objetivo.
- A equipe é fixa ou variável.
 - Se o tamanho da equipe varia ou os membros podem entrar e sair, deve-se evitar procedimentos complexos, pois novos membros podem não ter experiência.
- Os procedimentos a serem adotados não devem adicionar cargas cognitivas elevadas.

Enfim, o uso do *git* deve ter como objetivo facilitar o desenvolvimento do trabalho de forma produtiva e com rastreabilidade, sem adicionar uma carga de trabalho importante.

1.15.1 Fluxo de trabalho centralizado

Quem já usou o *cvs* ou *svn* costuma trabalhar com o conceito de um repositório único centralizado. Cada usuário baixa o repositório, faz alterações e depois envia para o servidor central.

É possível trabalhar com o *git* usando este modelo. Vai funcionar como no *cvs* ou *svn*, mas não estaremos aproveitando as potencialidades e funcionalidades que o *git* fornece.

Etapas:

- Cria-se o repositório e então cada usuário executa os comandos abaixo.
- Baixa uma cópia.
 - `git clone`
- Modifica ou adiciona novos arquivos.
 - `git add arquivos novos ou modificados`

- Como outros usuários podem ter modificado arquivos realizada a atualização dos dados locais.
 - `git pull`
 - neste caso se outros usuários tiverem feito modificações precisamos confirmar que esta tudo ok (confirmar a mesclagem).
- Uma alternativa é usar:
 - `git pull --rebase origin main`.
 - A opção `--rebase origin main`, informa que os commits locais devem ser colocados depois dos comites realizados pelos outros usuários.
 - Imagine que antes de modificar tínhamos a configuração: 1->2->3. Que o usuário Carlos fez o commit 4. Que você também fez o commit 4. Quando você der o comando `git pull`, teremos os dois comites 4 alinhados, a opção `--rebase` faz com que o seu comite receba o número 5, vindo depois do commit do Carlos.
- Se houver conflitos:
 - Tem de resolver os conflitos.
 - Adicionar os arquivos que tiveram conflito

```
* git add <some-file>
```

- e pular para próxima mensagem

```
* git rebase --continue
```

- Se por um acaso você se perder e esquecer o que estava fazendo cancele tudo e volte com o comando

```
* git rebase --abort.
```

- Tealiza o commit

```
- git commit -m"mensagem"
```

- Envia o commit para o servidor

- `git push origin main`

- origem: se refere ao repositório que foi clonado (ao que está no servidor)

- main: se refere ao banch que esta sendo sincronizado. Ou seja, estamos pedindo para que o servidor fique igual ao que temos localmente.

O fluxo de trabalho centralizado é indicado para:

- usuários com pouca experiência,
- equipes pequenas,
- quando os sistemas a serem desenvolvidos são simples.

1.15.2 Fluxo de trabalho baseado em funcionalidades

Neste caso o trabalho da equipe é dividido em funcionalidades a serem implementadas pelos diferentes membros da equipe.

- De forma ideal estas funcionalidades são independentes umas das outras.
 - Crie funcionalidades que possam ser integradas ao ramo principal de forma simples, ou seja, evite modelar funcionalidades que misturam muitas conceitos diferente classes.

Etapas

- Baixa uma cópia.
 - `git clone`

- Cria um ramo ou alterna para ele.
 - `git checkout main`
 - `git checkout -b nomeDaFuncionalidade`
 - ou
 - `git checkout -b nomeDaFuncionalidade main`
- Modifica ou adiciona novos arquivos e faz *commits*.
 - `git add arquivos novos ou modificados`
 - `git commit -m "nome ou descrição da mudança"`
- Envie as alterações para o repositório central (também funciona como um *backup*)
 - `git push -u origin nomeDaFuncionalidade`
 - A opção `-u origin` informa que é um ramo em cima da *origin*.
- Como os arquivos agora estão no servidor, outros usuários poderão ver como esta a nova funcionalidade antes mesmo da mesma ser finalizada.

- Poderão fazer comentários.
- Alterna para o ramo *main* e atualiza o mesmo
 - `git checkout main`
 - `git pull`
 - Corrige conflitos
- Mistura o ramo da funcionalidade criada com o ramo principal
 - `git pull origin nomeDaFuncionalidade`
 - `git push`

1.15.3 Todo - Doing - Done - Review - Aproved

Ao usar um sistema de gestão considere contruir as abas:

- TODO: a fazer
- DOING: fazendo

- DONE: feito/concluído (já foi testado, passou nos testes do desenvolvedor)
- REVIEW: revisado por outros usuários
- APROVED: aprovada a versão final;.

1.16 Diferenças entre *cvs*, *svn* e *git*

Veja na Tabela [1.3](#) uma relação entre comandos do *cvs*, do *svn* e do *git*.

Tabela 1.3: Relação entre comandos do *cvs* do *svn* e do *git*

<i>cvs</i> comando	<i>svn</i> command	<i>git</i> command
cvs -d repositorio get nomeProjeto	svn co repositorio/trunk nomeProjeto	git
cvs update -dP	svn update	git update
cvs add nome	svn add nome	git add nome
rm -f nome cvs remove nome	svn remove nome	git remove nome
mv nome novoNome; cvs remove nome; cvs add novoNome	svn move nome novoNome	git move nome novoNome
cvs commit	svn commit	git commit
cvs diff	svn diff	git diff
cvs -d repos get -r foo nomeProjeto	svn co repos/branches/foo nomeProjeto ; svn co repos/releases/foo	

1.17 Sentenças para o *git*

- Quando você passa como argumento de algum comando do *cvs* um nome de projeto, todos os arquivos do diretório do projeto e seus sub-diretórios sofrem o efeito do comando.
- Uma nome de projeto e o nome de um módulo são equivalentes. Ou seja, se o programa *cvs* espera um nome de projeto, você também pode passar o nome de um módulo.
- Um *tag*, um *release* e uma versão são equivalentes. Se o *cvs* espera um número de versão, você pode passar o nome do *tag*, ou o nome do *release*.
- O comando *export* é semelhante ao comando *checkout*, só que com *export* não é criado o sub-diretório CVS no diretório de trabalho. O comando *export* é usado quando se deseja uma cópia do sistema que vai ser distribuída (não sendo necessário os arquivos administrativos do *cvs*).
- Você pode simular um comando *cvs update* com a opção -n.

cvs -n update

- Durante o processo de *backup* do repositório é interessante impedir o acesso ao repositório (para garantir a integridade dos dados do *backup*). Para impedir o acesso ao repositório basta criar um arquivo chamado *cvs.lock* no diretório

raiz do repositório.

- Em alguns casos, podemos emitir um *cvs commit* com uma mensagem de *log* errada. Para substituir uma mensagem de *log* de uma determinada versão, use o comando abaixo:

Protótipo:

```
cvs admin -mN:"Mensagem"
```

Exemplo:

```
cvs admin -m1.3:"nova mensagem de log para versão 1.3"
```

- Se um arquivo binário foi erroneamente submetido ao repositório, o mesmo pode ser retirado usando-se um comando administrativo.

– `cvs admin -kb nomeDoArquivo`

- O arquivo CVSROOT/checkoutlist contém o nome dos arquivos novos que devem ser mantidos pelo *cvs*. Observe após o nome do arquivo (ex: readers), uma mensagem que será enviada para o usuário se o arquivo não for encontrado.

Exemplo:

```
readers Não foi possível obter o arquivo readers
cvsignore Não foi possível obter o arquivo cvsignore
```

- Outras operações com o arquivo CVSROOT/modules:
 - É possível obter a lista dos apelidos criados no arquivo modules.

cvs checkout -c

- Você pode criar um apelido para um sub-diretório do projeto, de forma que um comando *checkout* irá baixar apenas o sub-diretório.

```
apelido projeto/sub-diretorio
```

- Você pode criar um único apelido para dois projetos (-a indica mais de um projeto).

```
apelido -a projeto_1 projeto_2
```

- Num comando *cvs checkout* você pode especificar que não quer determinado sub-diretório usando a opção !.

```
apelido -a !diretorio_a_desconsiderar projeto
```

- Pode-se definir diferentes formas de acesso a arquivos, autenticações e sistemas de segurança. Veja o manual de configuração do *cvs*.
- Links úteis para usuários do *cvs*: <http://www.cvshome.org/>, a casa do *cvs*.
- Você encontrará informações adicionais sobre o *cvs* em [Cederqvist, 2006, Nolden and Kdevelop-Team, 2005, Hughs and Hughes, 1997, Wall, 2001].

1.18 Resumo do capítulo

Neste capítulo vimos o programa *git*. Como conseguir o *git*, suas características e versões, e algumas diferenças entre o *cvs*, o *svn* e o *git*.

1.19 Exercícios

1. Baixar e instalar o *git*.

2. Ler o manual do usuário.
3. Montar os programa do Capítulo ?? - Controle de Versões - O Programa *cvs*, utilizando o *git*.

Referências Bibliográficas

[Cederqvist, 2006] Cederqvist, P. (2006). *Version Management with CVS*. Free Software Foundation.

[Hughes and Hughes, 1997] Hughes, C. and Hughes, T. (1997). *Object Oriented Multithreading using C++: architectures and components*, volume 1. John Wiley Sons, 2 edition.

[Nolden and Kdevelop-Team, 2005] Nolden, R. and Kdevelop-Team (2005). *The User Manual to KDevelop*. kdevelop team, 3.3 edition.

[Proiete, 2011] Proiete, C. (2011). Git: Controlo de versões para pequenos e grandes projectos. In *Revista Portugal a Programar*, volume 29. ISSN 1647-0710.

[Wall, 2001] Wall, K. (2001). *Linux Programming Unleashed*, volume 1. SAMS, 2 edition.

Índice Remissivo

Comandos do git, [31](#)

cvs - anotações, [100](#)

cvs - branching, [86](#)

cvs - como criar release's, [80](#)

cvs - como recuperr arquivos, [84](#)

cvs - como ver as diferencas entre arquivos, [71](#)

cvs - histórico das alterações, [96](#)

cvs - mensagens de log, [97](#)

cvs - merging, [86](#)

cvs - misturas, [86](#)

cvs - ramos, [86](#)

cvs - status dos arquivos, [101](#)

cvs -como atualizar os arquivos locais, [68](#)

git - como adicionar/remover arquivos e diretórios, [58](#)

git - como atualizar o repositório, [54](#)

git - como criar tag's, [77](#)

git - desenvolvimento colaborativo, [74](#)

git - entendendo as versões, [66](#)

git - estado do repositório, [96](#)

git - releases, [77](#)

git - tag's, [77](#)

git-comandos, [31](#)

git-school - comandos, [117](#)